

รายงานโครงงานประจำรายวิชา Computer Networks

การพัฒนาเว็บไซต์ประชาสัมพันธ์หลักสูตร และระบบเฝ้าระวังความปลอดภัยบนคลาวด์

จัดทำโดย

นายกิตติธร	อุดร	รหัสนิสิต	67160060
นางสาวปุณยวีร์	บุญอุ้ม	รหัสนิสิต	67160075
นายกวิน	ทัศนพอดี	รหัสนิสิต	67160083
นายก้องภพ	นุตเจริญกุล	รหัสนิสิต	67160084
นางสาวกัญญาณัฐ	มาทุมมี	รหัสนิสิต	67160086
นายวรวิช	สายบัว	รหัสนิสิต	67160105
นางสาวสุธาทิพย์	มุกประดับทอง	รหัสนิสิต	67160109
นางสาวแก้วกาญจน์	สิงห์โตทอง	รหัสนิสิต	67160250

รายงานนี้เป็นส่วนหนึ่งของวิชา Computer Networks
หลักสูตรปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาการสารสนเทศ มหาวิทยาลัยบูรพา
ปีการศึกษา 2569

สารบัญ

บทที่ 1 บทนำ....1
1.1 ที่มาและความสำคัญ....1
1.2 วัตถุประสงค์....2
1.3 ขอบเขตของโครงงาน....3
1.4 ประโยชน์ที่คาดว่าจะได้รับ....4
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง....5
2.1 Cloud Computing และ Azure VPS....5
2.2 Ubuntu และ Web Stack....5
2.3 Nginx, PHP-FPM, MySQL และ WordPress....6
2.4 DNS, DuckDNS, HTTPS และ Let's Encrypt....6
2.5 SSH Public Key และ TOTP MFA....7
2.6 Firewall, iptables, ipset และ Fail2Ban....7
2.7 Reverse Proxy และ Rate Limiting....8
2.8 Security Logging และการวิเคราะห์ภัยคุกคามด้วย Gemini API....8
2.9 systemd สำหรับการทำงานอัตโนมัติ....9
บทที่ 3 การออกแบบและพัฒนาระบบ....10
3.1 ภาพรวมสถาปัตยกรรม....10
3.2 องค์ประกอบและหน้าที่ของระบบ....11
3.3 เครื่องมือ....12
3.4 การสร้าง Azure VPS และตั้งค่าเครือข่าย....13
3.5 การติดตั้ง Web Stack และพัฒนาเว็บไซต์ WordPress....14
3.6 การตั้งค่า Domain, HTTPS และการต่ออายุใบรับรองอัตโนมัติ....14
3.7 การรักษาความปลอดภัย SSH ด้วย Public Key, TOTP และ Fail2Ban....15
3.8 การตั้งค่า Firewall และจำกัดการเข้าถึงเฉพาะ IP ประเทศไทย....16

3.9 การตั้งค่า Reverse Proxy และ Rate Limiting 17
3.10 แนวทางตรวจสอบบริการ HTTPS 18
3.11 การจัดเก็บและประมวลผล Security Log 18
3.12 การวิเคราะห์ภัยคุกคามด้วย Gemini API 18
3.13 การพัฒนา Security Dashboard 20
3.14 การบล็อก IP อัตโนมัติและปลดบล็อกด้วย OTP 21
3.15 การแจ้งเตือนผ่านอีเมล 21
บทที่ 4 ผลการดำเนินงานและการทดสอบ 22
4.1 วิธีการทดสอบ 22
4.2 ผลการพัฒนาเว็บไซต์ประชาสัมพันธ์หลักสูตร 22
4.3 ผลการตรวจสอบระบบ Domain, HTTPS และ Reverse Proxy 23
4.4 ผลการทดสอบ SSH, MFA และ Fail2Ban 25
4.5 การป้องกัน Brute Force ด้วย Fail2Ban 26
4.6 ผลการทดสอบ Firewall และการจำกัด IP ตามประเทศ 26
4.7 ผลการทดสอบ Rate Limiting และ Security Logging 29
4.8 ผลการวิเคราะห์และการแสดงผลบน Security Dashboard 31
4.9 การตรวจสอบความปลอดภัยด้วย Cloudflare Security-Audit Skill 33
บทที่ 5 สรุปผลและข้อเสนอแนะ 39
5.1 สรุปผลการดำเนินโครงงาน 39
5.2 ปัญหา อุปสรรค และแนวทางแก้ไข 40
5.3 ข้อจำกัดของระบบ 42
5.4 แนวทางพัฒนาต่อ 43
บรรณานุกรม 44

สารบัญตาราง

ตารางที่ 3.1 องค์ประกอบและหน้าที่ของระบบ 11
ตารางที่ 3.2 เครื่องมือตามบันทึกโครงงาน 12
ตารางที่ 3.3 ทรัพยากรเครือข่ายบน Azure 13
ตารางที่ 3.4 พารามิเตอร์ Fail2Ban สำหรับ SSH 16
ตารางที่ 3.5 นโยบายพอร์ตของระบบ 17
ตารางที่ 3.6 นโยบาย Rate Limit 18
ตารางที่ 3.7 โครงสร้างโมดูลของระบบวิเคราะห์ 19
ตารางที่ 3.8 โครงสร้างผลลัพธ์การวิเคราะห์ 20
ตารางที่ 3.9 หน่วยบริการและรอบการทำงานของ systemd 21
ตารางที่ 4.1 รายละเอียดการทดสอบ 22
ตารางที่ 4.2 ผลการตรวจสอบระบบพื้นฐาน 23
ตารางที่ 4.3 ผลการทดสอบการยืนยันตัวตน SSH 26
ตารางที่ 4.4 ผลการทดสอบ Firewall 27

สารบัญภาพ

รูปที่ 3.1 สถาปัตยกรรมของระบบ 10
รูปที่ 3.2 การเชื่อมต่อจาก Windows ด้วย Private Key 13
รูปที่ 3.3 การติดตั้ง Nginx, PHP-FPM และ MySQL 14
รูปที่ 3.4 การสร้างฐานข้อมูลและผู้ใช้สำหรับ WordPress 14
รูปที่ 3.5 ผลการตรวจสอบค่าความปลอดภัย SSH และ TOTP 15
รูปที่ 3.6 คำสั่งตั้งค่า ipset และ iptables สำหรับ IP ประเทศไทย 16
รูปที่ 4.1 หน้าแรกของเว็บไซต์ประชาสัมพันธ์หลักสูตร 23
รูปที่ 4.2 ผลการตรวจสอบระบบทั้ง 7 ส่วน 24
รูปที่ 4.3 ผลการทดสอบ Reverse Proxy 25
รูปที่ 4.4 ผลการตรวจสอบการตั้งค่าการยืนยันตัวตน SSH 25
รูปที่ 4.5 ผลการทดสอบจากต่างประเทศผ่าน check-host.net พอร์ต 443 28
รูปที่ 4.6 ผลการทดสอบจากต่างประเทศผ่าน check-host.net พอร์ต 22 28
รูปที่ 4.7 ตัวอย่าง Security Log แบบ JSON 29
รูปที่ 4.8 ตัวอย่าง Security Log แยกตาม area 30
รูปที่ 4.9 ผลการทดสอบ Rate Limit (HTTP 429) 30
รูปที่ 4.10 หน้าภาพรวมของ Security Dashboard 31
รูปที่ 4.11 การวิเคราะห์ภัยคุกคามและรายการเหตุการณ์ 32
รูปที่ 4.12 ข้อมูลบัญชีผู้ใช้ที่ถูกเปิดเผยผ่าน WordPress REST API ก่อนการปรับปรุง 34
รูปที่ 4.13 ผลการตรวจสอบความปลอดภัยเว็บไซต์และหน้าบ้าน 35
รูปที่ 4.14 ผลการตรวจสอบด้านระบบและเครือข่าย 36
รูปที่ 4.15 ผลการตรวจสอบสิทธิ์การเข้าถึงและความเป็นส่วนตัว 37
รูปที่ 4.16 ผลการตรวจสอบ URL และจุดรั่วไหลของระบบ 38

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ในปัจจุบันเว็บไซต์เป็นช่องทางหลักในการประชาสัมพันธ์ข้อมูลขององค์กรและสถาบันการศึกษา การนำเว็บไซต์ขึ้นให้บริการบนอินเทอร์เน็ตจึงไม่ได้มีเพียงการติดตั้งระบบจัดการเนื้อหาให้ใช้งานได้เท่านั้น แต่ผู้ดูแลระบบต้องเข้าใจการทำงานของเครือข่าย การกำหนดชื่อโดเมน การเข้ารหัสการสื่อสาร และการป้องกันการโจมตีรูปแบบต่าง ๆ ที่เกิดขึ้นตลอดเวลา เช่น การเดารหัสผ่านแบบ Brute Force การสแกนหาช่องโหว่ การพยายามเข้าถึงไฟล์สำคัญ และการส่งคำขอปริมาณมากเพื่อให้ระบบหยุดให้บริการ โครงงานนี้จึงจัดทำขึ้นเพื่อออกแบบและติดตั้งเว็บไซต์ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์บนเครื่องเซิร์ฟเวอร์เสมือน (Virtual Private Server: VPS) ของ Microsoft Azure โดยใช้ Ubuntu Server ร่วมกับ Nginx, PHP-FPM, MySQL และ WordPress พร้อมทั้งเสริมความปลอดภัยหลายชั้น ได้แก่ การยืนยันตัวตน SSH ด้วยกุญแจสาธารณะร่วมกับรหัสผ่านใช้ครั้งเดียว (TOTP) การใช้ Firewall จำกัดการเข้าถึงเฉพาะหมายเลข IP ของประเทศไทย การใช้ Fail2Ban ป้องกันการเดารหัสผ่าน และการใช้ HTTPS ผ่านใบรับรองดิจิทัลจาก Let's Encrypt นอกจากนี้ ในระยะที่ 2 ของโครงงานได้ต่อยอดระบบให้มีความสามารถในการตรวจจับภัยคุกคามโดยแยก Nginx เป็น Frontend และ Backend ด้วยหลักการ Reverse Proxy จัดเก็บ Security Log ในรูปแบบ JSON กำหนด Rate Limit แยกตามประเภทของ Endpoint และนำ Google Gemini API มาวิเคราะห์ Log ร่วมกับ Security Rules เพื่อจำแนกประเภทการโจมตีและระดับความเสี่ยง จากนั้นแสดงผลผ่าน Security Dashboard พร้อมระบบบล็อก IP อัตโนมัติและการแจ้งเตือนทางอีเมล

1.2 วัตถุประสงค์

จากที่มาและความสำคัญข้างต้น การนำเว็บไซต์ขึ้นให้บริการบนอินเทอร์เน็ตอย่างปลอดภัยต้องอาศัยทั้งการติดตั้งระบบที่ถูกต้อง การควบคุมการเข้าถึงในระดับเครือข่ายและระดับเครื่อง และการเฝ้าระวังภัยคุกคามอย่างต่อเนื่อง คณะผู้จัดทำจึงกำหนดวัตถุประสงค์ของโครงงานให้ครอบคลุมตั้งแต่การติดตั้งเว็บไซต์ การเสริมความปลอดภัยให้เซิร์ฟเวอร์ ไปจนถึงการพัฒนาระบบวิเคราะห์และตอบสนองต่อภัยคุกคามอัตโนมัติ ดังนี้

1. เพื่อติดตั้งเว็บไซต์ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์ด้วย WordPress บน Azure VPS ให้เข้าถึงได้ผ่าน HTTPS และโดเมนของ DuckDNS

2. เพื่อเพิ่มความปลอดภัยในการเข้าถึงเซิร์ฟเวอร์ผ่าน SSH ด้วยการปิด Root Login ปิดการยืนยันตัวตนด้วยรหัสผ่าน และใช้ Public Key ร่วมกับ TOTP MFA

3. เพื่อกำหนด Firewall ให้รับการเชื่อมต่อเฉพาะจากหมายเลข IP ของประเทศไทย และป้องกันการโจมตีแบบ Brute Force ด้วย Fail2Ban

4. เพื่อปรับสถาปัตยกรรมเว็บเซิร์ฟเวอร์ด้วย Reverse Proxy และ Rate Limiting ให้ปลอดภัยและมีประสิทธิภาพมากขึ้น

5. เพื่อพัฒนาระบบวิเคราะห์ Security Log อัตโนมัติด้วย Google Gemini API ร่วมกับ Security Rules และแสดงผลผ่าน Security Dashboard

6. เพื่อให้ระบบสามารถตอบสนองต่อภัยคุกคามได้อัตโนมัติ ทั้งการบล็อก IP และการแจ้งเตือนผู้ดูแลระบบ

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตระยะที่ 1 (Stage 1)

- สร้าง VPS บน Microsoft Azure ติดตั้ง Ubuntu Server 24.04 LTS พร้อม Nginx, PHP-FPM, MySQL และ WordPress
- ใช้โดเมน networkkeroro.duckdns.org และให้บริการผ่าน HTTPS ด้วยใบรับรองจาก Let's Encrypt
- SSH ต้องปิด Root Login ปิด Password Authentication และใช้ PKI (SSH Key) เพิ่มเติมด้วย TOTP MFA และ Fail2Ban
- ติดตั้ง Firewall ด้วย iptables และ ipset เพื่อจำกัดการเข้าถึงพอร์ต 22, 80 และ 443 เฉพาะ IP ประเทศไทย
- พัฒนาเว็บไซต์ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์ด้วย WordPress

1.3.2 ขอบเขตระยะที่ 2 (Stage 2)

- ใช้ Reverse Proxy แยก Nginx Frontend (พอร์ต 443) ออกจาก Backend (127.0.0.1:8080)
- จัดเก็บ Security Log ในรูปแบบ JSON และจำแนกพื้นที่ของคำขอ
- กำหนด Rate Limit แยกตามประเภท Endpoint
- วิเคราะห์ Log อัตโนมัติด้วย Google Gemini API ร่วมกับ Data Masking, Validator, Security Rules และ Fallback
- แสดงผลผ่าน Security Dashboard พร้อมระบบบล็อก IP อัตโนมัติ ปลดบล็อกด้วย OTP และแจ้งเตือนทางอีเมล
- ควบคุมการทำงานอัตโนมัติทั้งหมดด้วย systemd

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- ได้เว็บไซต์ประชาสัมพันธ์หลักสูตรที่ใช้งานได้จริงและเข้าถึงได้อย่างปลอดภัยผ่าน HTTPS
- ได้ความรู้และทักษะการติดตั้ง ดูแล และเสริมความปลอดภัยเซิร์ฟเวอร์บนระบบคลาวด์
- ได้ระบบรักษาความปลอดภัยแบบหลายชั้น (Defense in Depth) ที่ลดโอกาสการถูกโจมตี
- ได้ระบบตรวจจับและตอบสนองต่อภัยคุกคามอัตโนมัติ ที่สรุปผลเป็นภาษาไทยให้ผู้ดูแลระบบเข้าใจได้ทันที
- ได้แนวทางการนำ Generative AI มาประยุกต์ใช้ในงานด้านความปลอดภัยเครือข่าย

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 Cloud Computing และ Azure VPS

Cloud Computing คือรูปแบบการให้บริการทรัพยากรคอมพิวเตอร์ เช่น หน่วยประมวลผล หน่วยความจำ พื้นที่จัดเก็บข้อมูล และเครือข่าย ผ่านอินเทอร์เน็ตตามความต้องการ ผู้ใช้ไม่จำเป็นต้องลงทุนจัดซื้อฮาร์ดแวร์เอง โดยแบ่งรูปแบบบริการหลักได้เป็น IaaS, PaaS และ SaaS โครงงานนี้ใช้บริการแบบ IaaS ซึ่งผู้ใช้ได้รับเครื่องเสมือนและต้องดูแลระบบปฏิบัติการและซอฟต์แวร์เอง

Virtual Private Server (VPS) คือเครื่องเซิร์ฟเวอร์เสมือนที่แบ่งทรัพยากรจากเครื่องจริงด้วยเทคโนโลยี Virtualization โดยแต่ละ VPS มีระบบปฏิบัติการและสิทธิ์ผู้ดูแลระบบเป็นของตนเอง Microsoft Azure เป็นผู้ให้บริการคลาวด์ของ Microsoft ที่มีบริการ Virtual Machine ให้เลือกขนาดตามความต้องการ โครงงานนี้ใช้ขนาด Standard_B2als_v2 ซึ่งเป็นเครื่องกลุ่ม Burstable เหมาะกับงานที่ใช้ CPU ไม่สูงต่อเนื่อง เช่น เว็บไซต์ขนาดเล็ก

2.2 Ubuntu และ Web Stack

Ubuntu Server เป็นระบบปฏิบัติการ Linux ที่นิยมใช้บนเซิร์ฟเวอร์ รุ่น LTS (Long Term Support) ได้รับการสนับสนุนด้านความปลอดภัยเป็นระยะเวลานาน โครงงานนี้ใช้ Ubuntu 24.04.5 LTS

บน Azure เครื่องเสมือนต้องอยู่ภายใน Virtual Network (VNet) ซึ่งเป็นเครือข่ายส่วนตัวเสมือนที่แยกออกจากผู้ใช้รายอื่น ภายใน VNet แบ่งเป็น Subnet เพื่อจัดกลุ่มทรัพยากร และควบคุมการรับส่งข้อมูลด้วย Network Security Group (NSG) ซึ่งทำหน้าที่เป็น Firewall ระดับเครือข่ายของ Azure โดยกำหนดกฎอนุญาตหรือปฏิเสธตามพอร์ต โปรโตคอล และที่อยู่ต้นทางปลายทาง การออกสู่อินเทอร์เน็ตของ Subnet ใช้ System Route เริ่มต้นของ Azure (0.0.0.0/0 ไปยัง Internet)

2.3 Nginx, PHP-FPM, MySQL และ WordPress

ระบบเว็บของโครงงานใช้ชุดซอฟต์แวร์ที่เรียกว่า LEMP Stack ประกอบด้วย Linux, Nginx, MySQL และ PHP

- Nginx เป็นเว็บเซิร์ฟเวอร์ที่ทำงานแบบ Event-driven รองรับการเชื่อมต่อพร้อมกันจำนวนมากโดยใช้หน่วยความจำต่ำ และสามารถทำหน้าที่เป็น Reverse Proxy, Load Balancer และ TLS Termination [3]
- PHP-FPM (FastCGI Process Manager) เป็นตัวจัดการกระบวนการของ PHP ที่ Nginx ส่งคำขอไปประมวลผลผ่านโปรโตคอล FastCGI
- MySQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ใช้เก็บข้อมูลเนื้อหา ผู้ใช้ และการตั้งค่าของ WordPress
- WordPress เป็นระบบจัดการเนื้อหา (CMS) แบบโอเพนซอร์สที่เขียนด้วย PHP มี Theme และ Plugin ให้เลือกใช้จำนวนมาก [4]

2.4 DNS, DuckDNS, HTTPS และ Let's Encrypt

Domain Name System (DNS) ทำหน้าที่แปลงชื่อโดเมนให้เป็นหมายเลข IP DuckDNS เป็นบริการ Dynamic DNS ฟรีที่ให้ชื่อโดเมนย่อยภายใต้ duckdns.org และรองรับการแก้ไข TXT Record ผ่าน API

HTTPS คือ HTTP ที่ทำงานบน TLS (Transport Layer Security) ซึ่งเข้ารหัสข้อมูลระหว่างเบราว์เซอร์กับเซิร์ฟเวอร์ และยืนยันตัวตนของเซิร์ฟเวอร์ด้วยใบรับรองดิจิทัล Let's Encrypt เป็นผู้ออกใบรับรองฟรีที่ใช้โปรโตคอล ACME ตรวจสอบความเป็นเจ้าของโดเมนก่อนออกใบรับรอง โดยมีวิธีตรวจสอบหลัก 2 แบบ ได้แก่

- HTTP-01 Challenge: Let's Encrypt เข้ามาอ่านไฟล์ยืนยันบนเว็บเซิร์ฟเวอร์ผ่านพอร์ต 80 ซึ่งต้องเปิดพอร์ต 80 ให้เซิร์ฟเวอร์ของ Let's Encrypt (อยู่ต่างประเทศ) เข้าถึงได้
- DNS-01 Challenge: ผู้ขอใบรับรองสร้าง TXT Record ที่ _acme-challenge ของโดเมน แล้ว Let's Encrypt ตรวจสอบผ่าน DNS โดยไม่ต้องเข้าถึงเซิร์ฟเวอร์โดยตรง จึงเหมาะกับระบบที่จำกัดการเข้าถึงด้วย Firewall

2.5 SSH Public Key และ TOTP MFA

SSH (Secure Shell) เป็นโปรโตคอลสำหรับเข้าควบคุมเครื่องเซิร์ฟเวอร์จากระยะไกลอย่างปลอดภัย การยืนยันตัวตนด้วย Public Key อาศัยหลักการโครงสร้างพื้นฐานกุญแจสาธารณะ (PKI) โดยเซิร์ฟเวอร์เก็บกุญแจสาธารณะ (Public Key) ส่วนผู้ใช้เก็บกุญแจส่วนตัว (Private Key) ไว้ การยืนยันตัวตนทำโดยพิสูจน์ว่าผู้ใช้ถือ Private Key ที่คู่กันโดยไม่ต้องส่งกุญแจผ่านเครือข่าย จึงปลอดภัยกว่าการใช้รหัสผ่าน

การยืนยันตัวตนหลายปัจจัย (Multi-Factor Authentication: MFA) คือการใช้หลักฐานมากกว่าหนึ่งประเภท TOTP (Time-based One-Time Password) ตามมาตรฐาน RFC 6238 สร้างรหัสตัวเลข 6 หลักจาก Secret ร่วมกับเวลาปัจจุบัน รหัสเปลี่ยนทุก 30 วินาที บน Linux ใช้โมดูล PAM (Pluggable Authentication Modules) ชื่อ pam_google_authenticator ร่วมกับแอป Google Authenticator บนโทรศัพท์

2.6 Firewall, iptables, ipset และ Fail2Ban

Firewall คือระบบควบคุมการรับส่งข้อมูลตามกฎที่กำหนด บน Linux ใช้ Netfilter ซึ่งจัดการผ่านคำสั่ง iptables โดยกฎจะถูกตรวจสอบตามลำดับใน Chain เช่น INPUT สำหรับข้อมูลที่เข้ามายังเครื่อง ลำดับของกฎจึงมีผลต่อผลลัพธ์โดยตรง

ipset เป็นเครื่องมือสร้างชุดของ IP หรือเครือข่าย (CIDR) ที่ค้นหาได้รวดเร็วด้วยโครงสร้างแบบ Hash ทำให้ iptables อ้างถึงชุด IP นับพันรายการได้ด้วยกฎเดียว เหมาะกับการจำกัดการเข้าถึงตามประเทศ (Geo-blocking) ซึ่งโครงงานนี้ใช้รายการ IP ประเทศไทยจาก IPdeny Country IP Blocks อย่างไรก็ตาม การจัดสรร IP ของแต่ละประเทศเปลี่ยนแปลงได้ และผู้ใช้สามารถเชื่อมต่อผ่าน VPN หรือ Proxy ได้ จึงต้องใช้ Geo-blocking ร่วมกับการยืนยันตัวตนและการตรวจสอบพฤติกรรม รวมถึงปรับปรุงรายการ IP ให้ทันสมัย

Fail2Ban เป็นโปรแกรมที่เฝ้าดู Log ของบริการต่าง ๆ เมื่อพบการพยายามยืนยันตัวตนล้มเหลวเกินจำนวนที่กำหนดภายในช่วงเวลาหนึ่ง จะเพิ่มกฎ Firewall เพื่อบล็อก IP นั้นชั่วคราว ช่วยป้องกันการโจมตีแบบ Brute Force ในโครงงานนี้ Fail2Ban จะเห็นเฉพาะคำขอที่ผ่านกฎ Firewall ประเทศไทยเข้ามาถึงบริการ SSH แล้วเท่านั้น

2.7 Reverse Proxy และ Rate Limiting

Reverse Proxy คือเซิร์ฟเวอร์ที่รับคำขอจากผู้ใช้แทนเซิร์ฟเวอร์ปลายทาง แล้วส่งต่อคำขอไปยัง Backend ภายใน ข้อดีคือซ่อนเซิร์ฟเวอร์ Backend จากอินเทอร์เน็ต รวมจุดจัดการ TLS นโยบายความปลอดภัย และการบันทึก Log ไว้ที่ Frontend จุดเดียว

Rate Limiting คือการจำกัดจำนวนคำขอต่อหน่วยเวลาจากผู้ใช้แต่ละราย Nginx ใช้โมดูล limit_req ซึ่งทำงานด้วยอัลกอริทึม Leaky Bucket โดยกำหนด rate และค่า burst สำหรับรองรับคำขอที่เข้ามาพร้อมกันชั่วขณะ เมื่อเกินกำหนดจะตอบกลับด้วยสถานะ 429 Too Many Requests การกำหนดค่าต้องคำนึงถึงการใช้งานปกติ และกรณีที่ผู้ใช้หลายคนออกอินเทอร์เน็ตผ่าน IP เดียวกัน (NAT) ซึ่งอาจถูกจำกัดพร้อมกัน

2.8 Security Logging และการวิเคราะห์ภัยคุกคามด้วย Gemini API

Security Logging คือการบันทึกเหตุการณ์ที่เกี่ยวข้องกับความปลอดภัย เช่น ที่อยู่ IP เส้นทางที่เรียก สถานะการตอบกลับ และ User-Agent การจัดเก็บในรูปแบบ JSON ทำให้โปรแกรมอ่านและประมวลผลต่อได้ง่าย การรวมข้อมูลตาม IP และช่วงเวลาเป็น Traffic Profile ช่วยเชื่อมโยงพฤติกรรมของหลายคำขอเข้าด้วยกัน เช่น จำนวนความพยายามเข้าสู่ระบบ จำนวนสถานะ 404 หรือ 429 และการเข้าถึงเส้นทางสำคัญ

Google Gemini API เป็นบริการโมเดลภาษาขนาดใหญ่ (Large Language Model: LLM) ของ Google ที่เรียกใช้ผ่าน API โครงงานนี้นำ Gemini มาวิเคราะห์พฤติกรรมของทราฟฟิกในเชิงบริบท เพื่อระบุประเภทการโจมตี ระดับความเสี่ยง เหตุผล และคำแนะนำ เนื่องจากคำตอบของโมเดลอาจคลาดเคลื่อนหรือไม่ตรงรูปแบบที่ร้องขอ ระบบจึงใช้ Validator ตรวจโครงสร้าง ใช้ Security Rules แบบกำหนดตายตัว (Deterministic) ตรวจผลซ้ำ และใช้เป็นระบบสำรองเมื่อ API ใช้งานไม่ได้ นอกจากนี้ ข้อมูลใน Log เป็นข้อมูลที่ผู้ใช้ภายนอกควบคุมได้ จึงต้องถือเป็นข้อมูลที่ไม่น่าเชื่อถือ การปิดบังข้อมูล (Masking) ช่วยลดการเปิดเผยข้อมูลสำคัญแต่ไม่ได้ป้องกัน Prompt Injection ได้ทั้งหมด จึงต้องจำกัดไม่ให้ผลจาก AI สั่งงานระบบโดยตรงโดยไม่ผ่านกฎตรวจสอบ

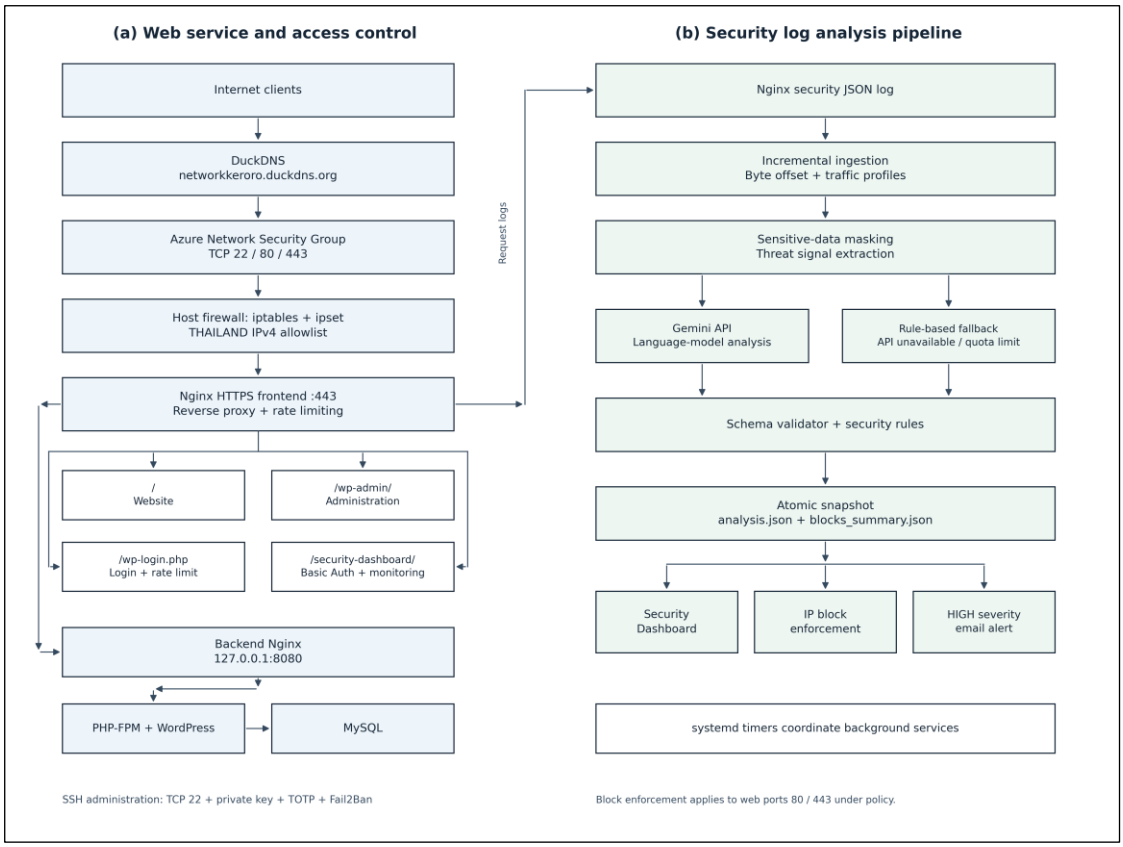
2.9 systemd สำหรับการทำงานอัตโนมัติ

systemd เป็นระบบจัดการบริการ (Service Manager) ของ Linux รุ่นใหม่ ใช้ไฟล์ Unit กำหนดบริการ (.service) และตัวตั้งเวลา (.timer) ซึ่งทำหน้าที่คล้าย cron แต่มีความสามารถมากกว่า เช่น การบันทึก Log ผ่าน journald การเริ่มทำงานใหม่เมื่อเกิดข้อผิดพลาด และการตั้งเวลาระดับวินาที การแยกแต่ละหน้าที่เป็น Unit ของตนเองช่วยให้ตรวจสอบสถานะและปัญหาได้ทีละส่วน ทั้งนี้ รอบเวลาของ Timer คือกำหนดการเรียกใช้งาน ไม่ใช่เวลาตอบสนองที่รับประกันเพราะยังขึ้นกับเวลาประมวลผลการเรียก API และภาระงานของเครื่อง

บทที่ 3 การออกแบบและพัฒนาระบบ

3.1 ภาพรวมสถาปัตยกรรม

ระบบออกแบบตามหลักการป้องกันหลายชั้น (Defense in Depth) แบ่งออกเป็น 2 ส่วนหลัก ได้แก่ ส่วนให้บริการเว็บและควบคุมการเข้าถึง และส่วนวิเคราะห์ Security Log ดังรูปที่ 3.1



รูปที่ 3.1 สถาปัตยกรรมของระบบ (a) ส่วนให้บริการเว็บและควบคุมการเข้าถึง (b) ส่วนวิเคราะห์ Security Log

จากรูปที่ 3.1 (a) แสดงเส้นทางของคำขอจากผู้ใช้ เมื่อผู้ใช้เรียกชื่อโดเมน networkkeroro.duckdns.org ระบบ DNS ของ DuckDNS จะแปลงชื่อเป็น Public IP ของ VPS จากนั้นการเชื่อมต่อต้องผ่าน Azure Network Security Group ซึ่งเปิดเฉพาะพอร์ต TCP 22, 80 และ 443 แล้วผ่าน Firewall ภายในเครื่อง (iptables ร่วมกับ ipset) ที่อนุญาตเฉพาะ IPv4 ในชุด THAILAND เมื่อผ่านแล้ว Nginx Frontend ที่พอร์ต 443 จะทำหน้าที่ TLS Termination, Reverse Proxy และ Rate Limiting โดยจำแนกคำขอเป็น 4 เส้นทาง ได้แก่ หน้าเว็บไซต์ (/), หน้าผู้ดูแล (/wp-admin/), หน้าเข้าสู่ระบบ (/wp-login.php) ซึ่งมี Rate Limit เข้มงวด และ Security Dashboard (/security-dashboard/) ซึ่งป้องกันด้วย Basic Authentication คำขอเว็บไซต์ถูกส่งต่อไปยัง Backend Nginx ที่ 127.0.0.1:8080 ซึ่งรับการเชื่อมต่อ

เฉพาะภายในเครื่อง แล้วส่งให้ PHP-FPM ประมวลผล WordPress และเข้าถึงฐานข้อมูล MySQL ส่วนผู้ดูแลระบบเข้าเครื่องผ่าน SSH พอร์ต 22 ซึ่งต้องผ่านเงื่อนไข IP ประเทศไทย ยืนยันตัวตนด้วย Private Key ร่วมกับรหัส TOTP และมี Fail2Ban เฝ้าระวังการเดารหัสผ่าน

รูปที่ 3.1 (b) แสดงการทำงานของระบบเฝ้าระวัง ซึ่งทำงานคู่ขนานหลังจากรับคำขอแล้ว Nginx Frontend บันทึกทุกคำขอลง Security Log แบบ JSON จากนั้น Pipeline อ่านเฉพาะข้อมูลใหม่ตามตำแหน่ง Byte Offset และรวมพฤติกรรมตาม IP เป็น Traffic Profile แล้วปิดบังข้อมูลสำคัญและสกัดสัญญาณภัยคุกคามเบื้องต้น ข้อมูลที่เตรียมแล้วจะถูกส่งให้ Gemini API วิเคราะห์เป็นหลัก หาก API ใช้งานไม่ได้หรือเกินโควตา ระบบจะสลับไปใช้ Rule-based Fallback แทน ผลลัพธ์จากทั้งสองทางต้องผ่าน Schema Validator และ Security Rules ก่อนถูกเขียนเป็น Snapshot (analysis.json และ blocks_summary.json) แบบ Atomic Write จากนั้นบริการ 3 ส่วนจะนำ Snapshot ไปใช้ ได้แก่ Security Dashboard สำหรับแสดงผล, ระบบบล็อก IP ซึ่งเพิ่ม IP ลงในกฎ Firewall สำหรับพอร์ต 80 และ 443 และระบบแจ้งเตือนทางอีเมลสำหรับเหตุการณ์ระดับ HIGH ทุกบริการถูกควบคุมการทำงานด้วย systemd timers ดังนั้น การบล็อก IP จึงมีผลกับคำขอถัดไปตามรอบการทำงานของ Enforcer ไม่ได้บล็อกคำขอที่เกิดขึ้นไปแล้ว

3.2 องค์ประกอบและหน้าที่ของระบบ

ตารางที่ 3.1 องค์ประกอบและหน้าที่ของระบบ

องค์ประกอบ	หน้าที่หลัก
DuckDNS	ชี้ชื่อโดเมนไปยัง Public IP ของ VPS
Azure NSG	ควบคุมทราฟฟิกในชั้นเครือข่ายคลาวด์
iptables และ ipset	จำกัดประเทศ พอร์ต และรายการ IP ที่ถูกบล็อก
Nginx Frontend	รับ HTTPS บันทึก Log และควบคุมอัตราคำขอ
Nginx Backend และ PHP-FPM	ให้บริการ WordPress ภายในเครื่อง
MySQL	จัดเก็บข้อมูลเว็บไซต์
Security Pipeline	รวมข้อมูล ปิดบังข้อมูล และวิเคราะห์ความเสี่ยง
Dashboard และ Enforcer	แสดงผลและบังคับใช้การบล็อกตามเงื่อนไข

3.3 เครื่องมือ

ตารางที่ 3.2 เครื่องมือตามบันทึกโครงงาน

รายการ	**ค่าที่ระบุ**
Cloud และขนาดเครื่อง	Azure Standard_B2als_v2
CPU และ RAM	2 vCPU และ RAM 4 GB
พื้นที่จัดเก็บ	SSD 30 GB
ระบบปฏิบัติการ	Ubuntu 24.04.5 LTS ตามเอกสารต้นทาง
Nginx และ PHP	Nginx 1.24.0 และ PHP 8.3.6
MySQL	8.0.46
WordPress	ข้อมูลต้นทางระบุทั้ง 7.1.2 และ 7.1.3 ต้องตรวจยืนยัน
Public IP	172.197.160.163
Domain	networkkeroro.duckdns.org

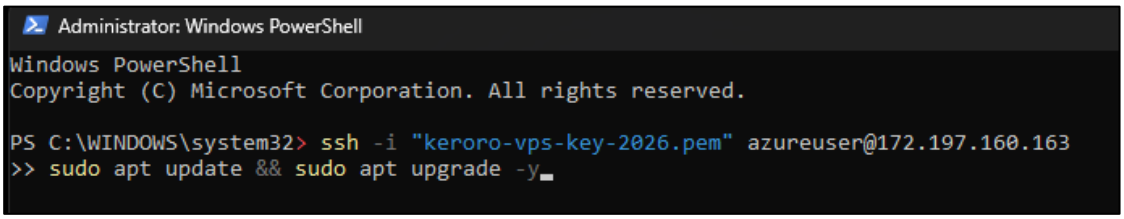
3.4 การสร้าง Azure VPS และตั้งค่าเครือข่าย

สร้าง Virtual Machine บน Azure เลือกระบบปฏิบัติการ Ubuntu 24.04 LTS และให้ Azure สร้างคู่กุญแจ SSH สำหรับผู้ใช้ azureuser จากนั้นกำหนดทรัพยากรเครือข่ายดังตารางที่ 3.3

ตารางที่ 3.3 ทรัพยากรเครือข่ายบน Azure

ทรัพยากร	ชื่อ / ค่า
Virtual Network (VNet)	wordpress-vnet
Subnet	wordpress-subnet
Network Security Group	wordpress-nsg
Route	0.0.0.0/0 ⟶ Internet (System Route)
Public IPv4	172.197.160.163
NSG Inbound	TCP 22 (SSH), 80 (HTTP), 443 (HTTPS)

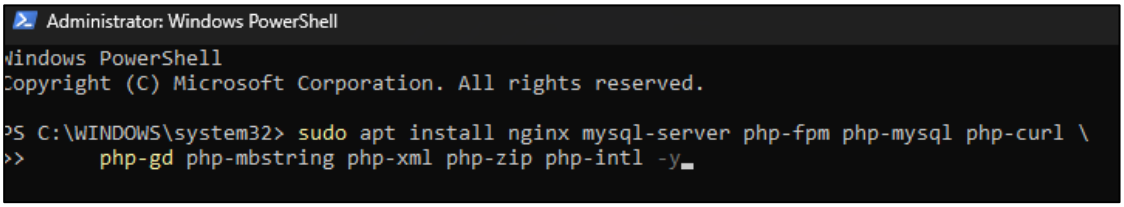
เมื่อสร้างเสร็จจึงเชื่อมต่อจากเครื่อง Windows ด้วย Private Key และปรับปรุงแพ็กเกจของระบบ



รูปที่ 3.2 การเชื่อมต่อจาก Windows ด้วย Private Key

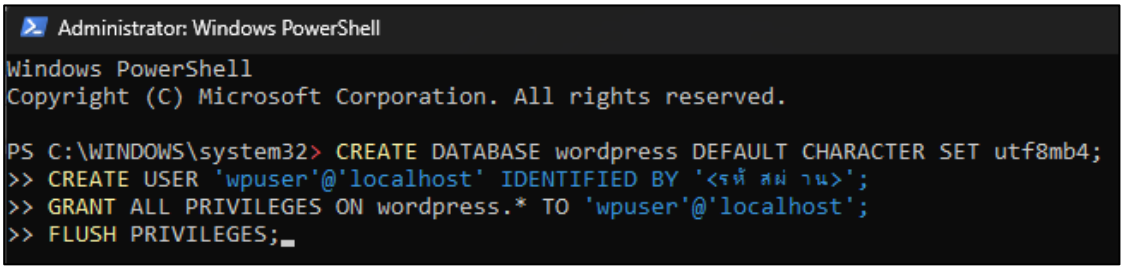
3.5 การติดตั้ง Web Stack และพัฒนาเว็บไซต์ WordPress

ติดตั้ง Nginx, PHP-FPM, ส่วนขยาย PHP ที่ WordPress ต้องใช้ และ MySQL Server



รูปที่ 3.3 การติดตั้ง Nginx, PHP-FPM และ MySQL

สร้างฐานข้อมูลและผู้ใช้เฉพาะสำหรับ WordPress โดยกำหนดสิทธิ์เฉพาะฐานข้อมูลของตนเอง (Principle of Least Privilege)



รูปที่ 3.4 การสร้างฐานข้อมูลและผู้ใช้สำหรับ WordPress

3.6 การตั้งค่า Domain, HTTPS และการต่ออายุใบรับรองอัตโนมัติ

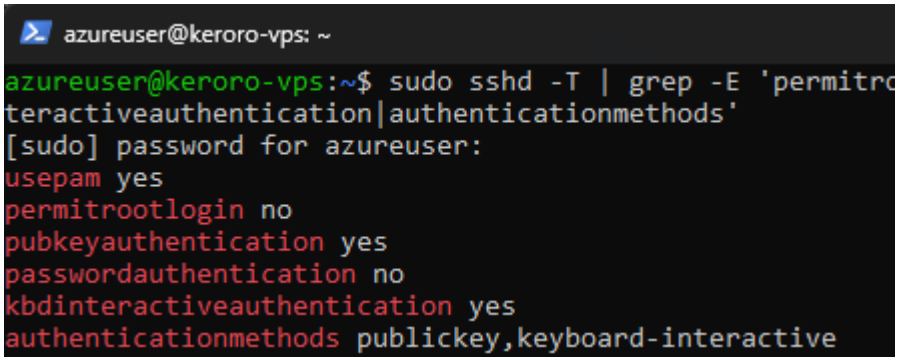
ลงทะเบียนโดเมนย่อย networkkeroro บน DuckDNS และชี้ไปยัง Public IP ของ VPS กำหนด server_name ใน Nginx ให้ตรงกับโดเมน จากนั้นขอใบรับรองจาก Let's Encrypt ด้วย Certbot โดยใช้กุญแจชนิด ECDSA

เนื่องจาก Firewall จำกัดพอร์ต 80 และ 443 ให้รับเฉพาะ IP ประเทศไทย เซิร์ฟเวอร์ตรวจสอบของ Let's Encrypt ซึ่งอยู่ต่างประเทศจึงไม่สามารถใช้ HTTP-01 Challenge ได้ คณะผู้จัดทำจึงเลือกใช้ DNS-01 Challenge โดยให้ Certbot สร้าง TXT Record ผ่าน API ของ DuckDNS ทำให้ต่ออายุใบรับรองได้อัตโนมัติโดยไม่ต้องเปิดพอร์ต 80 ให้ต่างประเทศ ไฟล์ที่เก็บ Token ของ DuckDNS ถูกกำหนดสิทธิ์ให้อ่านได้เฉพาะ root และตั้งให้ Nginx reload หลังต่ออายุสำเร็จ เพื่อให้ใช้ใบรับรองฉบับใหม่

3.7 การรักษาความปลอดภัย SSH ด้วย Public Key, TOTP และ Fail2Ban

3.7.1 SSH Hardening

แก้ไขไฟล์ /etc/ssh/sshd_config โดยสำรองไฟล์เดิมก่อนทุกครั้ง แล้วกำหนดค่าดังนี้ ค่า AuthenticationMethods บังคับให้ผู้ใช้ต้องผ่านทั้ง Public Key และ Keyboard Interactive (TOTP) จึงเข้าสู่ระบบได้



รูปที่ 3.5 ผลการตรวจสอบค่าความปลอดภัย SSH และ TOTP

3.7.2 TOTP MFA

ติดตั้ง libpam-google-authenticator และสร้าง Secret ให้ผู้ใช้ azureuser ด้วยคำสั่ง google-authenticator โดยเลือกใช้ Time-based Token ห้ามใช้รหัสซ้ำ ใช้ Time Window ค่าเริ่มต้น และเปิด Rate-limiting จากนั้นสแกน QR Code ด้วยแอป Google Authenticator

3.7.3 Fail2Ban

ติดตั้ง Fail2Ban และสร้างไฟล์ /etc/fail2ban/jail.local สำหรับ Jail ของ sshd ตามค่าในตารางที่ 3.4

ตารางที่ 3.4 พารามิเตอร์ Fail2Ban สำหรับ SSH

พารามิเตอร์	ค่าที่กำหนด
Jail	sshd
maxretry	5 ครั้ง
findtime	10 นาที
bantime	1 ชั่วโมง
backend	systemd
banaction	iptables-multiport

หมายความว่า IP ที่ยืนยันตัวตนล้มเหลวครบ 5 ครั้งภายใน 10 นาที จะถูกบล็อกเป็นเวลา 1 ชั่วโมง

3.8 การตั้งค่า Firewall และจำกัดการเข้าถึงเฉพาะ IP ประเทศไทย

ดาวน์โหลดรายการ IPv4 CIDR ของประเทศไทยจาก IPdeny (th.zone) แล้วนำเข้า ipset ชื่อ THAILAND ได้ทั้งหมด 1,203 รายการ จากนั้นกำหนดกฎ iptables ใน Chain INPUT

```
sudo ipset create THAILAND hash:net
for net in $(cat th.zone); do sudo ipset add THAILAND $net; done

iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
iptables -A INPUT -p icmp -j ACCEPT
iptables -A INPUT -p tcp --dport 22  -m set --match-set THAILAND src -j ACCEPT
iptables -A INPUT -p tcp --dport 80  -m set --match-set THAILAND src -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -m set --match-set THAILAND src -j ACCEPT
iptables -A INPUT -j REJECT
```

รูปที่ 3.6 คำสั่งตั้งค่า ipset และ iptables สำหรับ IP ประเทศไทย

ตารางที่ 3.5 นโยบายพอร์ตของระบบ

พอร์ตหรือบริการ	การเข้าถึง
TCP 22 SSH	IP ประเทศไทย พร้อม Public Key และ TOTP
TCP 80 HTTP	IP ประเทศไทย
TCP 443 HTTPS	IP ประเทศไทย
TCP 8080 Backend Nginx	เฉพาะ 127.0.0.1 ภายในเครื่อง
TCP 8000 Admin API	เฉพาะ 127.0.0.1 ภายในเครื่อง
Loopback, RELATED/ESTABLISHED, ICMP	ACCEPT
อื่น ๆ ทั้งหมด	REJECT

กฎบล็อก IP ที่เพิ่มโดยระบบเฝ้าระวัง (หัวข้อ 3.14) ถูกแทรกไว้ก่อนกฎอนุญาตพอร์ต 80 และ 443 เพื่อให้มีผลก่อน เนื่องจาก iptables ตรวจกฎตามลำดับ

เพื่อให้กฎคงอยู่หลังเริ่มระบบใหม่ ติดตั้ง iptables-persistent และ ipset-persistent แล้วบันทึกกฎด้วย netfilter-persistent save ซึ่งบริการ netfilter-persistent จะโหลด ipset และกฎ iptables ทุกครั้งที่บูตเครื่อง

3.9 การตั้งค่า Reverse Proxy และ Rate Limiting

3.9.1 Reverse Proxy

แยก Nginx เป็น 2 ชั้น โดย Frontend รับการเชื่อมต่อ HTTPS ที่พอร์ต 443 ทำหน้าที่ TLS Termination บังคับนโยบายความปลอดภัย Rate Limit และบันทึก Security Log จากนั้นส่งต่อคำขอไปยัง Backend Nginx ที่ผูกกับ 127.0.0.1:8080 ซึ่งรับการเชื่อมต่อเฉพาะภายในเครื่องเท่านั้น Backend เชื่อถือ Header X-Real-IP และ X-Forwarded-For เฉพาะเมื่อมาจาก 127.0.0.1 เท่านั้น เพื่อป้องกันผู้ใช้ภายนอกปลอม Header ให้ระบบบันทึก IP ผิด

3.9.2 Rate Limiting

กำหนด Policy แยกตามประเภท Endpoint แทนการใช้ค่าเดียวทั้งระบบ เพื่อให้ผู้ใช้ทั่วไปใช้งานได้ลื่นไหล แต่จำกัดการโจมตีในจุดสำคัญ

ตารางที่ 3.6 นโยบาย Rate Limit

Endpoint	อัตรา	**Burst**	หมายเหตุ
Website	8 req/s	30	-
WordPress Admin	12 req/s	40	-
WordPress Login	10 req/s	10	ป้องกัน Brute Force
Security DashBoard	10 req/min	10	ร่วมกับ Basic Auth
Static Files	ยกเว้น	-	CSS, JS, รูปภาพ, ฟอนต์, ไอคอน
analysis.json	ยกเว้น	-	ไม่บันทึก Log เพื่อกัน Self-Polling

การยกเว้น analysis.json มีเหตุผลเพราะ Dashboard เรียกไฟล์นี้ซ้ำทุกไม่กี่วินาทีเพื่ออัปเดตข้อมูล หากถูกบันทึกและนำไปวิเคราะห์ จะสร้างเหตุการณ์ของตัวเองเพิ่มขึ้นเรื่อย ๆ อย่างไรก็ตาม ไฟล์นี้ยังคงอยู่ภายใต้ Basic Authentication เช่นเดียวกับ Dashboard

3.10 แนวทางตรวจสอบบริการ HTTPS

3.11 การจัดเก็บและประมวลผล Security Log

กำหนด log_format แบบ JSON (escape=json) ใน Nginx และบันทึกลงไฟล์ /var/log/nginx/security_access.log และ security_error.log โดยแต่ละบรรทัดมีฟิลด์ time, client_ip, area, method, uri, status, bytes, request_time, referer และ user_agent การใช้ escape=json ทำให้อักขระพิเศษจากผู้ใช้ไม่ทำให้โครงสร้าง JSON เสียหรือแตกบรรทัด ฟิลด์ area ใช้ map ของ Nginx จำแนกคำขอเป็น website, login, admin และ security-dashboard เพื่อให้วิเคราะห์แยกพื้นที่ได้ง่าย

3.12 การวิเคราะห์ภัยคุกคามด้วย Gemini API

พัฒนาโปรแกรมวิเคราะห์ด้วยภาษา Python ไว้ที่ /opt/security-ai/ แบ่งเป็นโมดูลตามหน้าที่ ดังตารางที่ 3.7

ตารางที่ 3.7 โครงสร้างโมดูลของระบบวิเคราะห์

ไฟล์	หน้าที่
security_pipeline.py	ควบคุม Pipeline อ่าน Log และประสานงานโมดูล
gemini_client.py	เรียก Gemini API แบบหลาย Key สลับโมเดล พร้อม Retry/Backoff
signals.py	สกัดสัญญาณภัยคุกคามด้วย Pre-compiled Regex
masking.py	ปิดบังข้อมูลสำคัญก่อนส่งให้ LLM
validator.py	ตรวจโครงสร้างผลลัพธ์ด้วย Pydantic V2 และ Security Rules
fallback.py	ตรวจจับแบบ Rule-based เมื่อ API ใช้งานไม่ได้
cache.py	In-Memory TTL Cache อายุ 3,600 วินาที

3.12.1 การอ่าน Log และสร้าง Traffic Profile

Pipeline อ่านเฉพาะบรรทัดใหม่โดยจดจำตำแหน่ง Byte Offset ล่าสุด (Incremental Ingestion) จึงไม่ต้องอ่านไฟล์ซ้ำ และเมื่อพบว่าไฟล์ Log ถูกหมุน (Log Rotation) หรือมีขนาดเล็กกว่า Offset เดิมจะเริ่มอ่านจากต้นไฟล์ใหม่ เพื่อไม่ให้อ่านข้ามหรืออ่านซ้ำ จากนั้นรวมพฤติกรรมตาม Client IP ในช่วงเวลา 1–15 นาที เป็น Traffic Profile ที่ประกอบด้วยจำนวนคำขอ อัตราความถี่ สัดส่วนสถานะ 401/403/404/429/5xx จำนวนครั้งที่เรียก /wp-login.php หรือ /xmlrpc.php เส้นทาง Query String และ User-Agent

3.12.2 Data Masking และ Signal Extraction

ก่อนส่งข้อมูลให้ Gemini ระบบจะปิดบังรหัสผ่าน Token, JWT, อีเมล และข้อมูลส่วนบุคคลแบบ Fail-closed กล่าวคือหากปิดบังไม่สำเร็จจะไม่ส่งข้อมูลออกไป พร้อมกันนั้นใช้ Regex สแกนหาสัญญาณเบื้องต้น เช่น SQL Injection, XSS, Path Traversal, RCE, Scanner Bot และการเข้าถึงไฟล์สำคัญอย่าง .env หรือ wp-config.php.bak สัญญาณเหล่านี้ใช้เป็นหลักฐานประกอบการวิเคราะห์ ไม่ใช่ข้อสรุปว่าระบบถูกเจาะสำเร็จ

3.12.3 การเรียก Gemini API

ส่ง Traffic Profile ที่ผ่านการ Masking แล้วให้ Gemini วิเคราะห์ โดยใน Prompt กำหนดให้ถือข้อมูลจาก Log เป็นข้อมูลเท่านั้น ไม่ใช่คำสั่ง และให้ตอบกลับเป็น JSON ตาม Schema ที่กำหนด ระบบ

รองรับหลาย API Key แบบ Round-Robin และสลับระหว่างโมเดล gemini-3.5-flash-lite กับ gemini-3.8-flash เพื่อกระจายการใช้โควตา

ตารางที่ 3.8 โครงสร้างผลลัพธ์การวิเคราะห์

ฟิลด์	ค่าที่เป็นไปได้/ความหมาย
severity	LOW / MEDIUM / HIGH
attack_type	brute_force, flood_dos, sqli, xss, lfi_path_traversal, rce_webshell, sensitive_file_access, scanner_bot
reason_th	เหตุผลภาษาไทยสั้น กระชับ อิงหลักฐานจริง
evidence	สัญญาณที่ตรวจพบ เช่น sensitive_path:/.env, rate_limited_count
recommended_action	allow / rate_limit / block_ip / investigate

3.12.4 Validator, Security Rules และ Fallback

ผลลัพธ์จาก Gemini ทุกครั้งต้องผ่าน Validator ที่เขียนด้วย Pydantic V2 เพื่อตรวจชนิดข้อมูลและค่าที่อนุญาต หากผลไม่ถูกต้องจะถูกปฏิเสธ นอกจากนี้มี Discrepancy Protection ซึ่งห้ามไม่ให้ AI ลดระดับความเสี่ยงต่ำกว่าที่ Security Rules ตรวจพบ เช่น หาก Rule พบการเข้าถึง .env แต่ AI ให้ระดับ LOW ระบบจะยึดระดับจาก Rule เพื่อป้องกัน False Negative

หาก API ขัดข้อง เช่น เกินโควตา (HTTP 429) หรือเชื่อมต่อไม่ได้ ระบบจะสลับไปใช้ Deterministic Fallback Engine ที่ตรวจจับด้วยกฎทันที ร่วมกับ Cache ที่ลดการเรียก API ซ้ำ ทำให้ระบบวิเคราะห์ทำงานต่อได้ จากนั้นเขียนผลเป็น analysis.json และ blocks_summary.json แบบ Atomic Write (เขียนไฟล์ชั่วคราวแล้วเปลี่ยนชื่อทับ) เพื่อไม่ให้ Dashboard หรือ Enforcer อ่านไฟล์ที่เขียนไม่เสร็จ

3.13 การพัฒนา Security Dashboard

พัฒนา Dashboard ที่ https://networkkeroro.duckdns.org/security-dashboard/ ป้องกันด้วย HTTPS, Basic Authentication, Rate Limit และ Security Logging โดยโหลดข้อมูลจาก analysis.json แสดงผลแบบ Near Real-time ข้อมูลที่แสดงประกอบด้วย

- ตัวเลขสรุป: Total Requests, Unique IPs, Security Events, AI Anomalies และ High Severity
- สถานะ Engine (Gemini AI หรือ Fallback) และเวลาที่ข้อมูลถูกสร้างล่าสุด
- Top IP Addresses และ Recent Security Events พร้อมเหตุผลภาษาไทยและหลักฐาน

- สถานะ Block IP และกราฟสรุปด้านความปลอดภัย

3.14 การบล็อก IP อัตโนมัติและปลดบล็อกด้วย OTP

บริการ security-enforcer ทำงานแยกจาก Pipeline โดยอ่านรายการ IP ที่ผลวิเคราะห์แนะนำให้บล็อก แล้วปรับสถานะ (Reconcile) ให้ตรงกับ ipset สำหรับบล็อก ซึ่งมีกฎ iptables DROP ทราฟฟิกที่พอร์ต 80 และ 443 การแยกหน้าที่ช่วยให้ตรวจสอบได้ว่าความผิดพลาดเกิดจากการวิเคราะห์หรือการบังคับใช้กฎ และทุกการบล็อกจะบันทึก IP เวลา เหตุผล และกฎที่ใช้ไว้ เพื่อให้ผู้ดูแลตรวจสอบย้อนหลังและแก้ไขกรณี False Positive ได้ หากผู้ดูแลต้องการปลดบล็อก สามารถกดจาก Dashboard ซึ่งจะเรียก security-admin-api ที่ 127.0.0.1:8000 ส่งรหัส OTP ไปทางอีเมลของทีม และปลดบล็อกเมื่อยืนยันรหัสถูกต้อง

3.15 การแจ้งเตือนผ่านอีเมล

บริการ security-email-alert ตรวจหาเหตุการณ์ระดับ HIGH แล้วส่งอีเมลแจ้งเตือนแบบ HTML ไปยังอีเมลของทีม ประกอบด้วย IP ประเภทการโจมตี เหตุผล และหลักฐาน โดยมีช่วงพัก (Cooldown) 5 นาที เพื่อไม่ให้ส่งอีเมลซ้ำจำนวนมาก

ตารางที่ 3.9 หน่วยบริการและรอบการทำงานของ systemd

Unit	รอบการทำงาน	หน้าที่
security-pipeline.timer	ทุก 5 วินาที	อ่าน Log ใหม่ วิเคราะห์ เขียน Snapshot
security-enforcer.timer	ทุก 5 วินาที	ซิงก์ IP ที่ต้องบล็อกเข้า ipset/iptables
security-email-alert.timer	ทุก 30 วินาที	ส่งอีเมลแจ้งเตือนเหตุการณ์ HIGH
security-archive.timer	ทุกวัน 02:30 (ตาม Timezone ของเครื่อง)	บีบอัดและเก็บ Log เก่า
security-admin-api.service	ทำงานตลอด	API ยืนยัน OTP สำหรับปลดบล็อก IP

บทที่ 4 ผลการดำเนินงานและการทดสอบ

บทนี้นำเสนอผลการพัฒนาระบบและผลการทดสอบแต่ละองค์ประกอบตามที่ออกแบบไว้ในบทที่ 3 เพื่อยืนยันว่าระบบทำงานได้ถูกต้องและเป็นไปตามข้อกำหนดของ Stage 1 และ Stage 2

4.1 วิธีการทดสอบ

การทดสอบระบบมีวัตถุประสงค์เพื่อยืนยันว่าแต่ละองค์ประกอบทำงานได้ตามที่ออกแบบไว้ คณะผู้จัดทำแบ่งการทดสอบออกเป็น 3 กลุ่ม ได้แก่ การทดสอบระบบพื้นฐานและบริการเว็บ การทดสอบมาตรการควบคุมการเข้าถึง และการทดสอบระบบเฝ้าระวังและตอบสนองอัตโนมัติ

4.1.1 รายละเอียดการทดสอบ

ตารางที่ 4.1 รายละเอียดการทดสอบ

รายการ	รายละเอียด
เครื่องเซิร์ฟเวอร์	Azure VPS (Ubuntu 24.04.5 LTS) ที่ 172.197.160.163
เครื่องทดสอบ	คอมพิวเตอร์ Windows ผ่าน PowerShell และเว็บเบราว์เซอร์
เครือข่ายในประเทศไทย	อินเทอร์เน็ตบ้าน 3BB (AIS)
เครื่องมือ	ssh, curl, เว็บเบราว์เซอร์, Google Authenticator

4.1.2 ขั้นตอนและเกณฑ์การทดสอบ

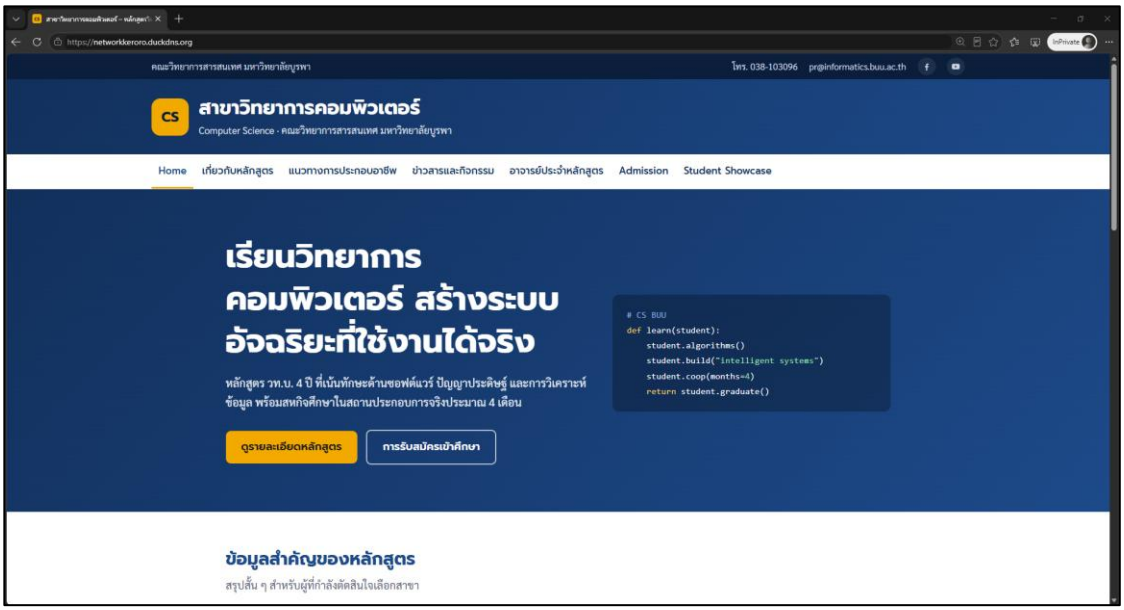
การทดสอบแต่ละกรณีดำเนินการตามขั้นตอนดังนี้

1. กำหนดกรณีทดสอบและผลที่คาดหวังจากการออกแบบในบทที่ 3
2. ทดสอบจากเครือข่ายต้นทางที่กำหนด และบันทึกวันเวลาที่ทดสอบ
3. ตรวจผลจากหน้าจอผู้ใช้ และตรวจซ้ำจากฝั่งเซิร์ฟเวอร์ด้วยคำสั่งตรวจสอบหรือ Log ที่เกี่ยวข้อง
4. เปรียบเทียบผลที่เกิดขึ้นจริงกับผลที่คาดหวัง และบันทึกภาพหน้าจอเป็นหลักฐาน

4.2 ผลการพัฒนาเว็บไซต์ประชาสัมพันธ์หลักสูตร

เว็บไซต์ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์ให้บริการที่ https://networkkeroro.duckdns.org ประกอบด้วยหน้าแรก เกี่ยวกับหลักสูตร เรียนอะไรบ้าง จุดเด่นของหลักสูตร อาชีพหลังเรียนจบ ข่าวสารและ

กิจกรรม และติดต่อเรา ผู้ดูแลจัดการเนื้อหาได้ผ่านหน้า /wp-admin และเว็บไซต์แสดงผลได้ทั้งบนคอมพิวเตอร์และมือถือ



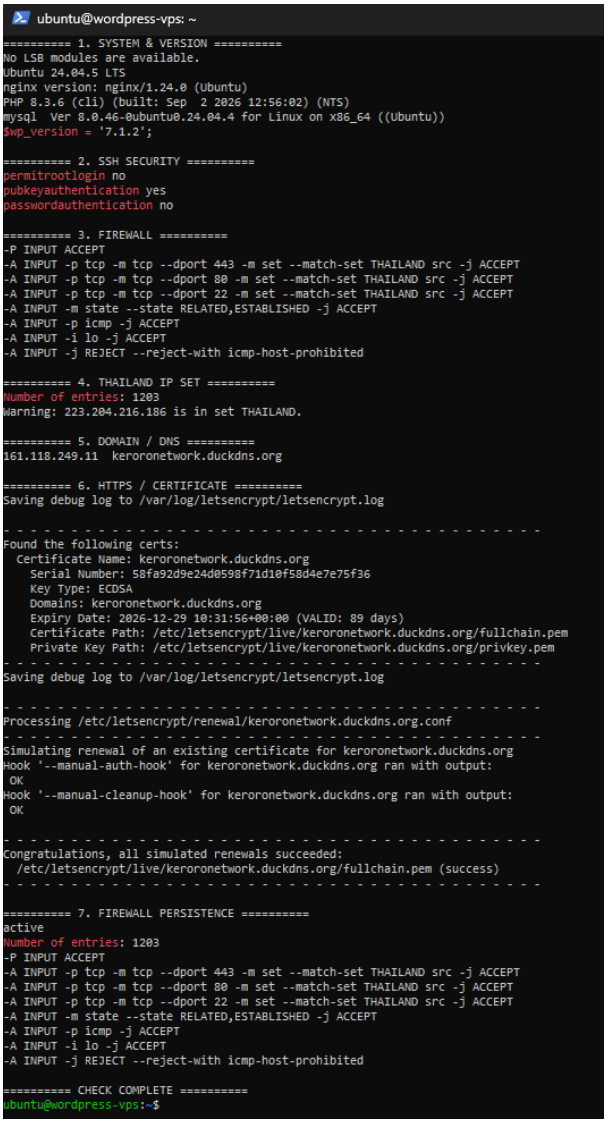
รูปที่ 4.1 หน้าแรกของเว็บไซต์ประชาสัมพันธ์หลักสูตร

4.3 ผลการตรวจสอบระบบ Domain, HTTPS และ Reverse Proxy

ตรวจสอบระบบด้วยชุดคำสั่งเดียวครอบคลุม 7 ส่วน ได้แก่ เวอร์ชันซอฟต์แวร์ การตั้งค่า SSH กฎ Firewall ชุด IP ประเทศไทย การชี้โดเมน ใบรับรอง HTTPS และการคงอยู่ของ Firewall (คำสั่งอยู่ในภาคผนวก ก.1) ได้ผลดังตารางที่ 4.2

ตารางที่ 4.2 ผลการตรวจสอบระบบพื้นฐาน

หัวข้อ	ผลที่ได้
ระบบปฏิบัติการ	Ubuntu 24.04.5 LTS
Nginx / PHP / MySQL	1.24.0 / 8.3.6 / 8.0.46
WordPress	7.1.3
DNS	networkkeroro.duckdns.org 172.197.160.163
Certificate	VALID (Let’s Encrypt, ECDSA)
Auto Renewal (dry-run)	Congratulations, all simulated renewals succeeded



รูปที่ 4.2 ผลการตรวจสอบระบบทั้ง 7 ส่วน

ผลการตรวจสอบแสดงว่าโดเมนชี้ไปยัง Public IP ของ VPS ได้ถูกต้อง ใบรับรอง HTTPS ใช้งานได้ และการจำลองต่ออายุผ่าน DNS-01 Challenge สำเร็จ จึงยืนยันได้ว่าใบรับรองต่ออายุอัตโนมัติได้โดยไม่ต้องเปิดพอร์ต 80 ให้ต่างประเทศ

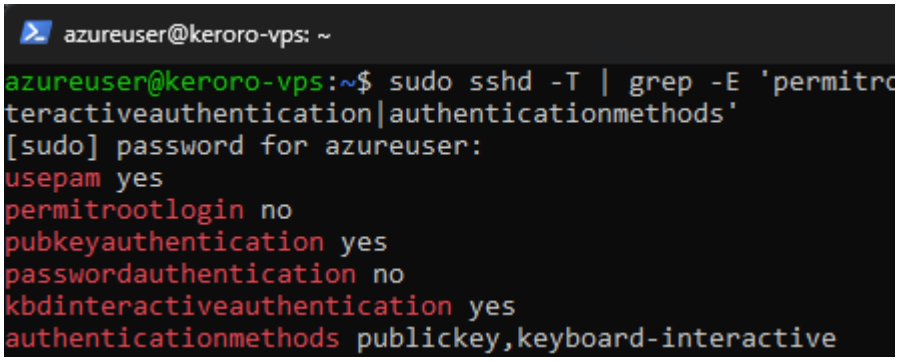
การทดสอบ Reverse Proxy พบว่า Backend Nginx รับการเชื่อมต่อที่ 127.0.0.1:8080 เท่านั้น และเมื่อเรียกผ่าน HTTPS Frontend ที่พอร์ต 443 คำขอถูกส่งต่อไปยัง Backend และ WordPress ตอบกลับสถานะ 200 OK สำเร็จ



รูปที่ 4.3 ผลการทดสอบ Reverse Proxy

4.4 ผลการทดสอบ SSH, MFA และ Fail2Ban

ตรวจสอบค่าการตั้งค่า SSH ได้ผลดังนี้



รูปที่ 4.4 ผลการตรวจสอบการตั้งค่าการยืนยันตัวตน SSH

จากนั้นทดสอบเปิด SSH Session ใหม่จาก Windows PowerShell ระบบยืนยัน Private Key แล้วถาม Verification code เมื่อใส่รหัส 6 หลักจาก Google Authenticator สามารถเข้าสู่ระบบได้โดยไม่ถามรหัสผ่าน Linux เพิ่ม ผลการทดสอบแต่ละกรณีแสดงในตารางที่ 4.3

ตารางที่ 4.3 ผลการทดสอบการยืนยันตัวตน SSH

กรณีทดสอบ	ผลที่คาดหวัง	ผลจริง
Private Key ถูก + TOTP ถูก	เข้าสู่ระบบได้	ผ่าน
หลังผ่าน Key และ TOTP แล้ว ไม่ถามรหัสผ่าน Linux เพิ่ม	ไม่ถาม	ผ่าน
Private Key ถูก + TOTP ผิด	ปฏิเสธ	ผ่าน (ระบบถาม Verification code ซ้ำและไม่อนุญาตให้เข้า)
ไม่มี Private Key (ใช้รหัสผ่าน)	ปฏิเสธ	ผ่าน (Permission denied (publickey) โดยไม่ถามรหัสผ่าน)
Login ด้วย root	ปฏิเสธ	ผ่าน (Permission denied (publickey))

ผลการทดสอบแสดงว่าการเข้าสู่ระบบต้องผ่านทั้ง Private Key และรหัส TOTP เท่านั้น เมื่อขาดปัจจัยใดปัจจัยหนึ่ง ระบบจะปฏิเสธการเชื่อมต่อทันที และกรณีที่ไม่มี Private Key ระบบไม่เปิดให้ใส่รหัสผ่านเลย เนื่องจาก AuthenticationMethods กำหนดให้ต้องผ่าน Public Key ก่อน นอกจากนี้ การเข้าสู่ระบบด้วยบัญชี root ถูกปฏิเสธตามค่า PermitRootLogin no

4.5 การป้องกัน Brute Force ด้วย Fail2Ban

ทดสอบ Fail2Ban โดยเชื่อมต่อ SSH ด้วยชื่อผู้ใช้ที่ไม่มีอยู่ในระบบซ้ำหลายครั้งจากเครือข่ายมือถือในประเทศไทย ซึ่งผ่านเงื่อนไข IP ประเทศไทยแล้ว ผลคือ [ผลจริง เช่น ครั้งที่ 1–5 ถูกปฏิเสธด้วย Permission denied และตั้งแต่ครั้งที่ 6 ไม่สามารถเชื่อมต่อได้ (Connection timed out)] แสดงว่า Fail2Ban บล็อก IP ดังกล่าวตามเงื่อนไข maxretry 5 ครั้งภายใน 10 นาที ขณะที่การเชื่อมต่อจากเครือข่ายอื่นยังใช้งานได้ตามปกติ

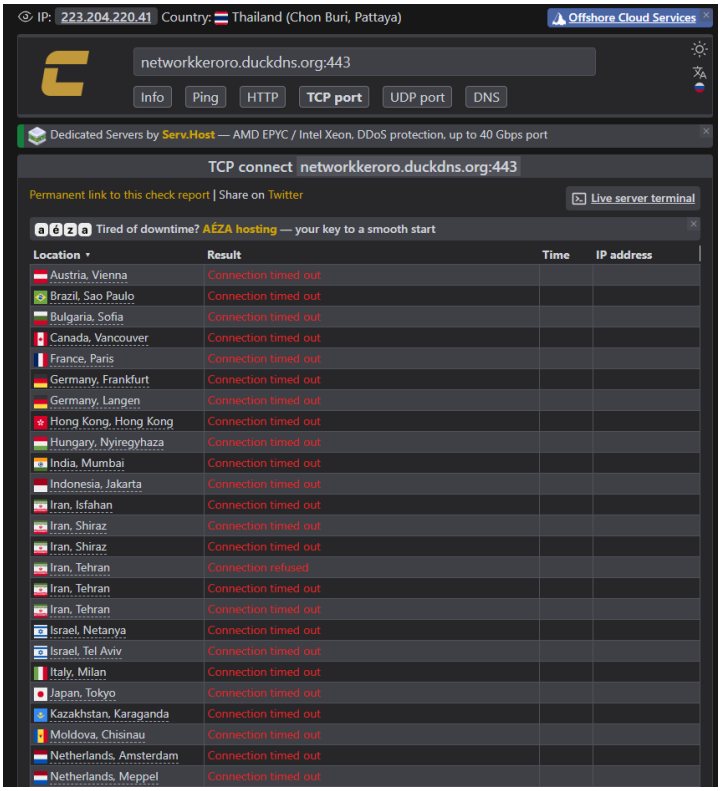
4.6 ผลการทดสอบ Firewall และการจำกัด IP ตามประเทศ

จากผลการตรวจสอบในรูปที่ 4.2 พบว่าชุด IP ชื่อ THAILAND มีจำนวน 1,203 รายการ กฎ iptables ของพอร์ต 22, 80 และ 443 อ้างอิงชุด THAILAND ตามที่ออกแบบ และบริการ netfilter-persistent อยู่ในสถานะ active

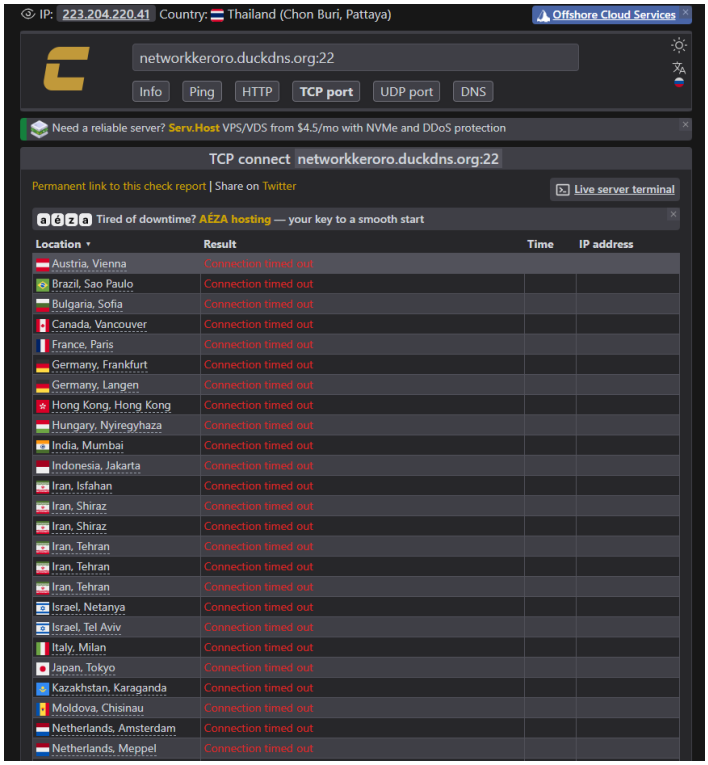
จากนั้นทดสอบการเชื่อมต่อจากเครือข่ายในประเทศไทย (อินเทอร์เน็ตบ้าน 3BB) และจากเครื่องตรวจสอบในต่างประเทศของเว็บไซต์ check-host.net ซึ่งทดลองเชื่อมต่อ TCP จากเซิร์ฟเวอร์กว่า 50 แห่งใน 30 ประเทศพร้อมกัน ได้ผลดังตารางที่ 4.4

ตารางที่ 4.4 ผลการทดสอบ Firewall

ต้นทาง	พอร์ต	ผลที่คาดหวัง	ผลจริง
IP ประเทศไทย (3BB)	443 (HTTPS)	เข้าได้	ผ่าน
IP ประเทศไทย + Private Key + TOTP	22 (SSH)	เข้าได้	ผ่าน
เครื่องตรวจสอบต่างประเทศ (check-host.net)	443	ถูกปฏิเสธ	ผ่าน
เครื่องตรวจสอบ ต่างประเทศ (check-host.net)	22	ถูกปฏิเสธ	ผ่าน



รูปที่ 4.5 ผลการทดสอบจากต่างประเทศผ่าน check-host.net พอร์ต 443

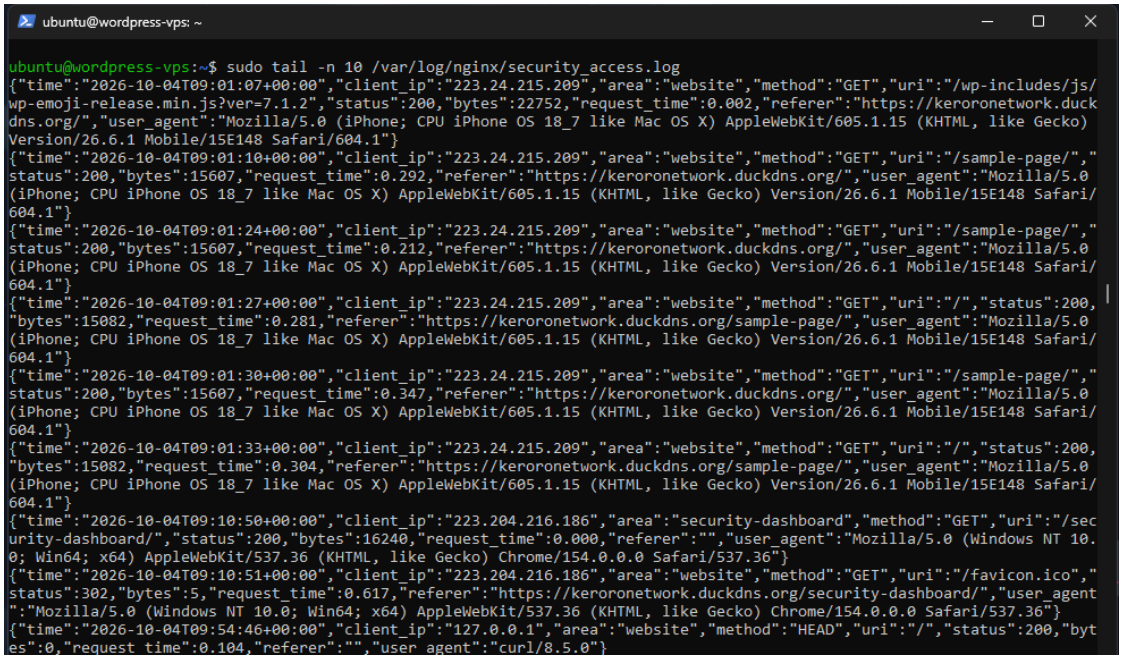


รูปที่ 4.6 ผลการทดสอบจากต่างประเทศผ่าน check-host.net พอร์ต 22

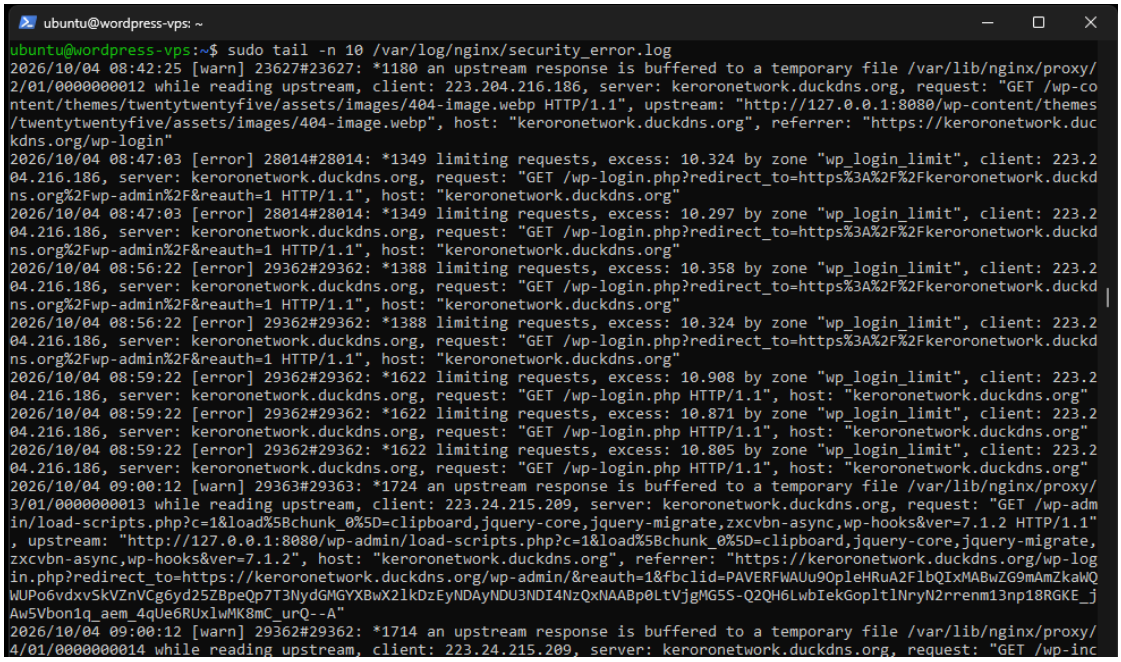
4.7 ผลการทดสอบ Rate Limiting และ Security Logging

ไฟล์ /var/log/nginx/security_access.log บันทึกคำขอในรูปแบบ JSON ครบทุกฟิลด์ตามที่ออกแบบ และจำแนก area เป็น website, login, admin และ security-dashboard ได้ถูกต้อง

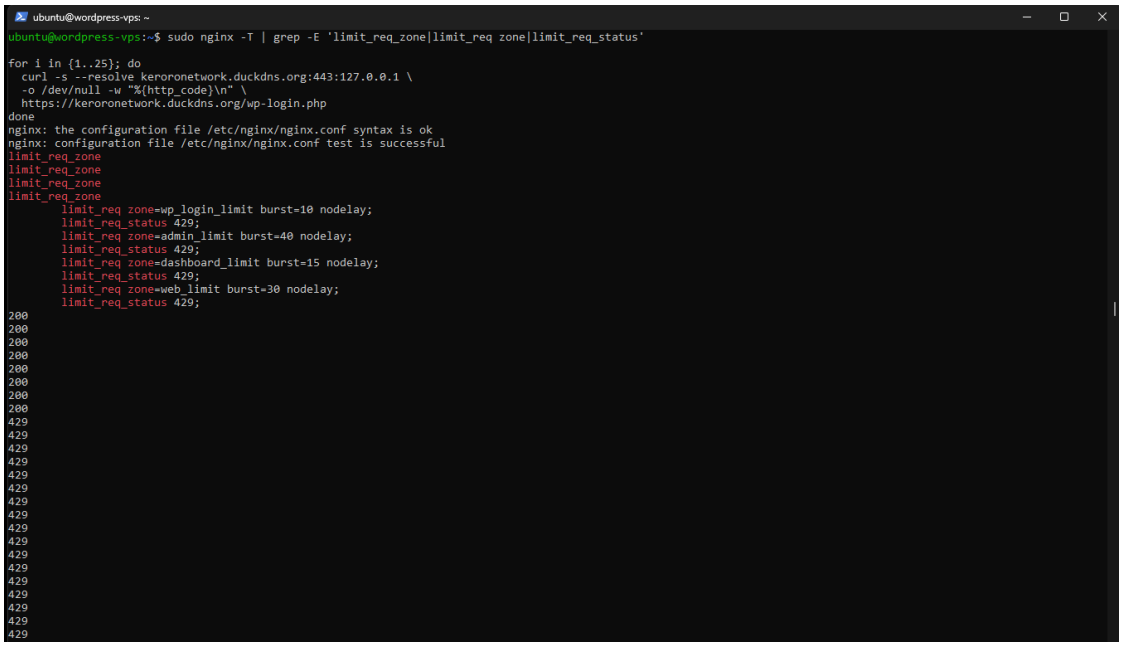
เมื่อทดสอบส่งคำขอไปยัง /wp-login.php ถี่เกินอัตราที่กำหนด ระบบตอบกลับด้วยสถานะ 429 Too Many Requests และสถานะดังกล่าวถูกบันทึกลง Security Log เพื่อนำไปวิเคราะห์ต่อ ขณะที่การเปิดหน้าเว็บและการโหลด Static Files ตามปกติไม่ถูกจำกัด แสดงว่าการแยก Policy ตาม Endpoint ทำงานได้ตามที่ออกแบบ



รูปที่ 4.7 ตัวอย่าง Security Log แบบ JSON



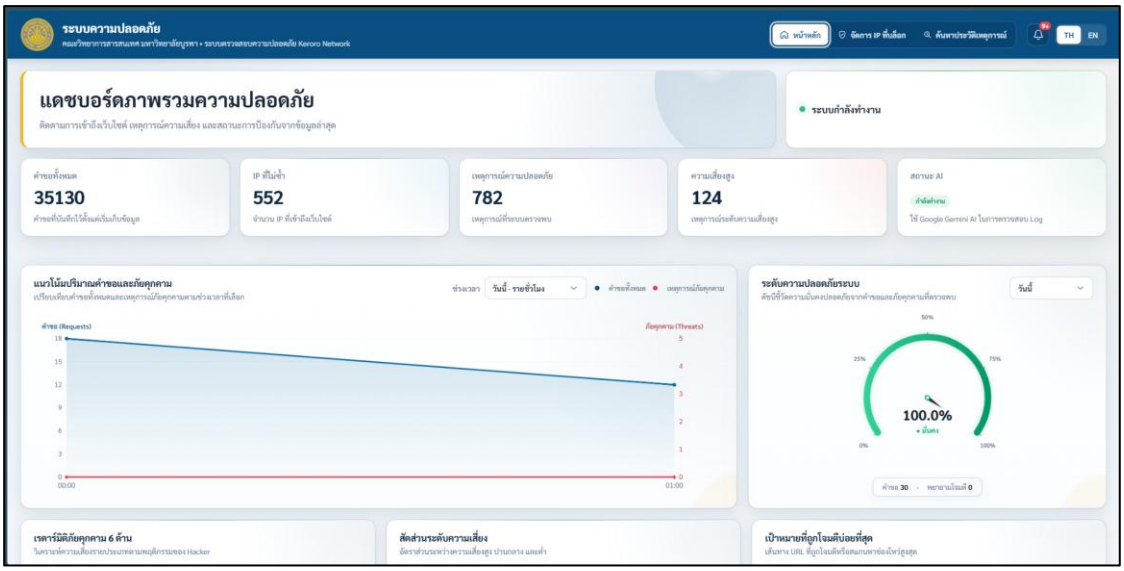
รูปที่ 4.8 ตัวอย่าง Security Log แยกตาม area



รูปที่ 4.9 ผลการทดสอบ Rate Limit (HTTP 429)

4.8 ผลการวิเคราะห์และการแสดงผลบน Security Dashboard

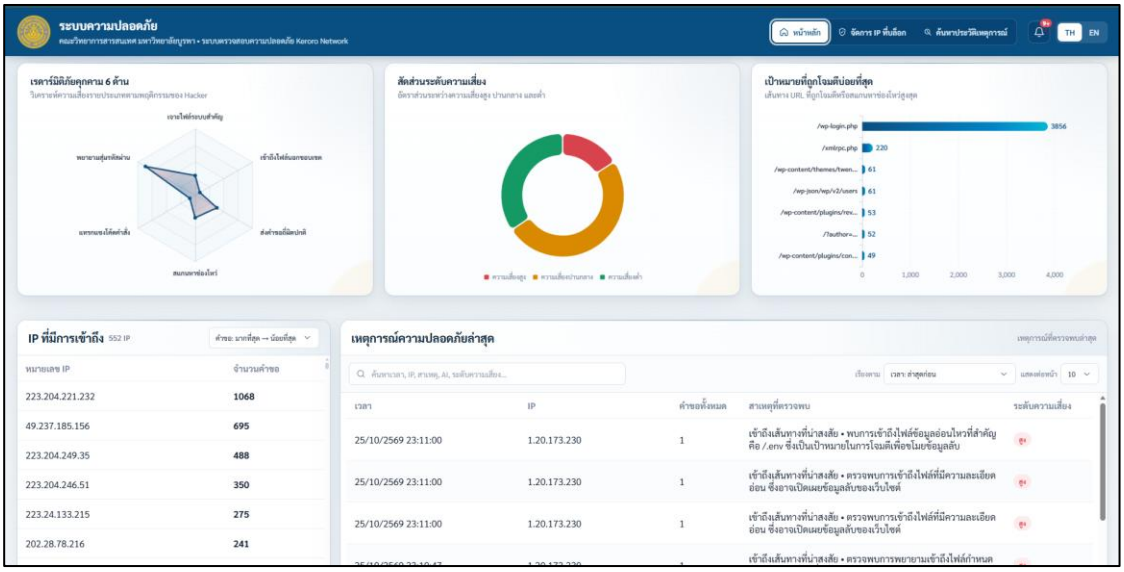
Security Dashboard แสดงสถานะ AI เป็น กำลังทำงาน โดยใช้ Google Gemini AI ในการตรวจสอบ Log แสดงว่า Security Pipeline ทำงานอัตโนมัติและเรียกใช้ Gemini API ได้ตามปกติ จากข้อมูลที่บันทึกตั้งแต่เริ่มเก็บข้อมูล ระบบมีคำขอทั้งหมด 35,130 คำขอ จาก IP ที่ไม่ซ้ำกัน 552 หมายเลข และตรวจพบเหตุการณ์ด้านความปลอดภัย 782 เหตุการณ์ ในจำนวนนี้เป็นระดับความเสี่ยงสูง 124 เหตุการณ์ ดังรูปที่ 4.10



รูปที่ 4.10 หน้าภาพรวมของ Security Dashboard

เมื่อวิเคราะห์ตามประเภทพฤติกรรม แผนภูมิเรดาร์แสดงว่าภัยคุกคามที่พบมากที่สุดคือการพยายามสุ่มรหัสผ่าน (Brute Force) สอดคล้องกับกราฟเป้าหมายที่ถูกโจมตีบ่อยที่สุด ซึ่งพบว่า /wp-login.php ถูกเรียก 3,856 ครั้ง และ /xmlrpc.php ถูกเรียก 220 ครั้ง ทั้งสองเส้นทางเป็นช่องทางเข้าสู่ระบบของ WordPress ที่ผู้โจมตีนิยมใช้เดารหัสผ่าน นอกจากนี้ยังพบการเรียก /wp-json/wp/v2/users และ /?author= ซึ่งเป็นเทคนิคค้นหาชื่อผู้ใช้ของ WordPress (User Enumeration) และการสแกนหา Theme และ Plugin ที่อาจมีช่องโหว่

ตารางเหตุการณ์ความปลอดภัยล่าสุดแสดงรายละเอียดแต่ละเหตุการณ์ ได้แก่ เวลา IP ต้นทาง สาเหตุที่ตรวจพบ และระดับความเสี่ยง ตัวอย่างเช่น ระบบตรวจพบการพยายามเข้าถึงไฟล์ /.env ซึ่งเป็นไฟล์ที่มักเก็บข้อมูลลับของระบบ และจัดเป็นความเสี่ยงระดับสูง พร้อมอธิบายเหตุผลเป็นภาษาไทยว่าเป็นการพยายามโจมตีเพื่อขโมยข้อมูลลับ ทำให้ผู้ดูแลเข้าใจเหตุการณ์ได้ทันทีโดยไม่ต้องอ่าน Raw Log ดังรูปที่ 4.11



รูปที่ 4.11 การวิเคราะห์ภัยคุกคามตามประเภท เป้าหมาย และเหตุการณ์ความปลอดภัย

ข้อสังเกตจากผลการวิเคราะห์ คือ เหตุการณ์จำนวนมากมาจาก IP ในประเทศไทย ซึ่งผ่านเงื่อนไข Firewall ได้ตามปกติ แสดงให้เห็นว่าการจำกัด IP ตามประเทศเพียงอย่างเดียวไม่เพียงพอต่อการป้องกัน จึงจำเป็นต้องมีการวิเคราะห์พฤติกรรม Rate Limit และการบล็อก IP อัตโนมัติร่วมด้วยตามหลัก Defense in Depth

ด้านการเข้าถึง Dashboard ป้องกันด้วย HTTPS และ Basic Authentication รองรับการสลับภาษาไทยและอังกฤษ และแสดงผลบนมือถือได้อย่างถูกต้อง

4.9 การตรวจสอบความปลอดภัยด้วย Cloudflare Security-Audit Skill

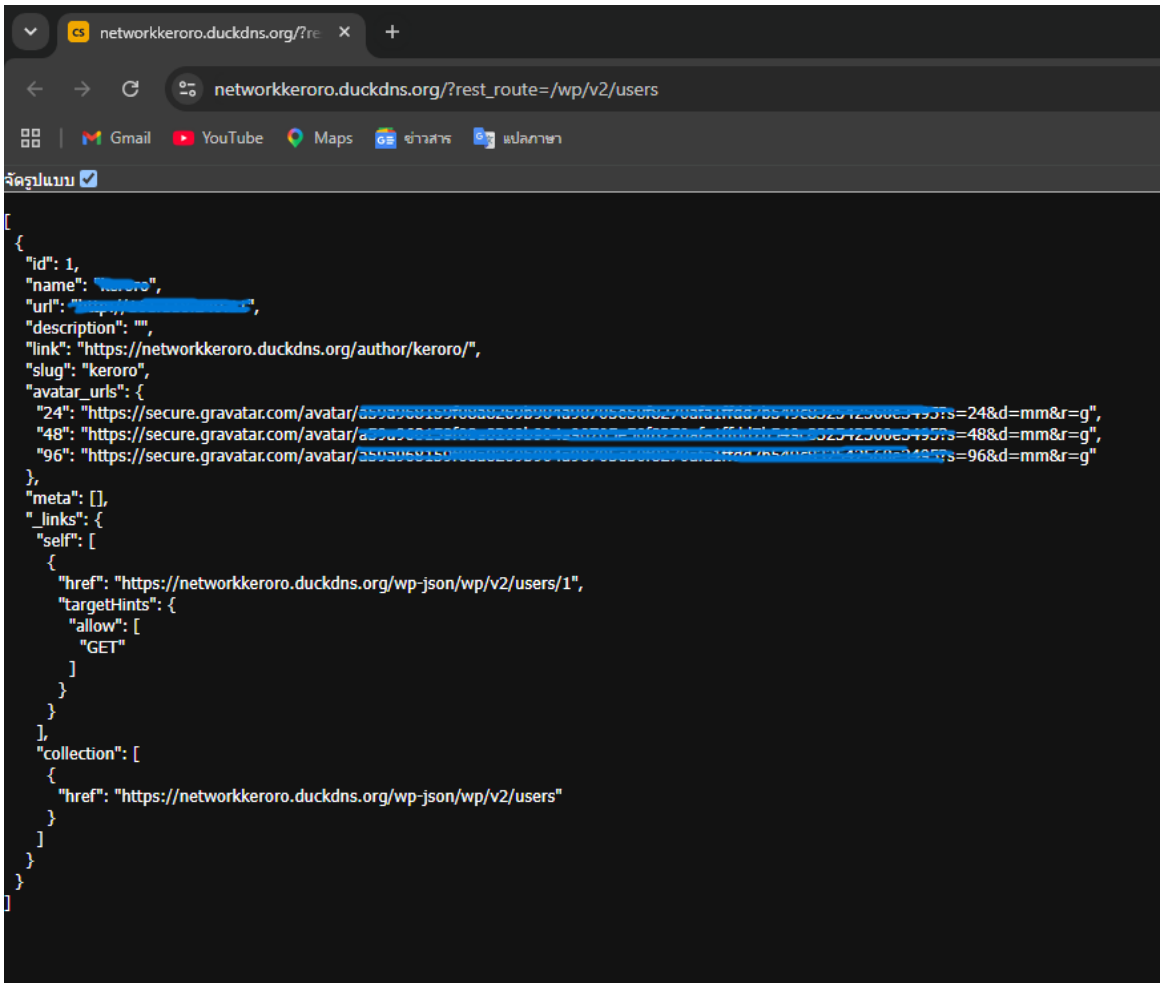
4.9.1 วัตถุประสงค์ของการตรวจสอบ

หัวข้อนี้ประเมินความปลอดภัยของเว็บไซต์และเซิร์ฟเวอร์ของโครงงาน โดยใช้ Cloudflare security-audit skill (https://github.com/cloudflare/security-audit-skill) เป็นเครื่องมือหลัก [17]

skill นี้ตรวจจากซอร์สโค้ดและไฟล์ตั้งค่าของระบบ ผลที่ได้คือรายการประเด็น (findings) และตารางความครอบคลุม (coverage ledger) ไม่ได้สแกนหรือโจมตีเว็บไซต์จริง ทีมนำผลจาก skill ไปจัดทำ checklist เทียบสภาพก่อนและหลังการปรับปรุงระบบ แบ่งเป็น 4 ด้าน ดังนี้

1. การปิดกั้น URL และจุดรั่วไหลของระบบ (URL & Attack Surface): URL และ Endpoint ที่เปิดให้เข้าถึงจากภายนอก
2. ความปลอดภัยเว็บไซต์และหน้าบ้าน (Web & Frontend): การเข้ารหัสข้อมูล การจัดการค่าลับ และการป้องกันช่องโหว่บนหน้าเว็บ
3. ความปลอดภัยระบบและเครือข่าย (System & Network): การเข้าถึง SSH, Firewall, สิทธิ์ของบริการ และการจัดเก็บ Log
4. สิทธิ์การเข้าถึงและความเป็นส่วนตัว (Access & Data Privacy): การเปิดเผยข้อมูลผู้ใช้และการปกป้องข้อมูลสำคัญ

ตัวอย่างจุดเสี่ยงที่พบก่อนการปรับปรุง คือการเปิดเผยข้อมูลบัญชีผู้ใช้ผ่าน WordPress REST API ดังรูปที่ 4.12



รูปที่ 4.12 ข้อมูลบัญชีผู้ใช้ที่ถูกเปิดเผยผ่าน WordPress REST API ก่อนการปรับปรุง

เมื่อเรียก URL https://networkkeroro.duckdns.org/?rest_route=/wp/v2/users โดยไม่ต้องล็อกอิน ระบบตอบกลับเป็นข้อมูล JSON ของบัญชีผู้ใช้ id 1 ได้แก่ชื่อผู้ใช้และ slug keroro เว็บไซต์ในโปรไฟล์ (url) ลิงก์หน้าผู้เขียน (link) และรูปโปรไฟล์ (avatar_urls) ผู้ไม่หวังดีจึงรู้ชื่อบัญชีแอดมินได้ทันที (User Enumeration) และเหลือเพียงการเดารหัสผ่าน ทำให้โจมตีแบบ Brute Force ได้ง่ายขึ้น ผลการตรวจสอบทั้งหมดก่อนและหลังการปรับปรุงแยกตามหมวดหมู่ แสดงในหัวข้อ 4.9.2

4.9.2 ผลการตรวจสอบก่อนและหลังการปรับปรุง

1) ความปลอดภัยเว็บไซต์ และ หน้าบ้าน (Web & Frontend)

#	จุดตรวจความปลอดภัยสำคัญ	ก่อนปรับปรุง (Before)	หลังปรับปรุง (After)
1	การเข้ารหัสเว็บ (HTTPS / TLS)	[x] บังคับ HTTPS และ TLS 1.2/1.3	[x] บังคับ HTTPS และ TLS 1.2/1.3
2	รหัสลับ WordPress (Salts)	[ ] ค่าเริ่มต้นซ้ำกัน เสี่ยงปลอมสิทธิ์	[x] สุ่มคีย์ความยาว 64 ตัวอักษรใหม่ 8 ชุด
3	กราฟ Dashboard (Stored XSS)	[ ] URL โชว์บนกราฟ เสี่ยงฝังโค้ดขโมยสิทธิ์	[x] ทำ Escape HTML ทุกจุด โค้ดแทรกไม่รัน
4	นโยบายสคริปต์เว็บ (CSP)	[ ] เปิด unsafe-inline ยอมรับโค้ดแทรก	[x] บังคับ script-src 'self' ปิดโค้ดแทรกถาวร
5	ป้องกันสคริปต์ในโฟลเดอร์รูปภาพ	[ ] เสี่ยงโดนแอบวาง WebShell ในรูปภาพ	[x] Nginx บล็อกคำขอไฟล์ PHP ทันที (HTTP 404)

รูปที่ 4.13 ผลการตรวจสอบด้านความปลอดภัยเว็บไซต์และหน้าบ้าน (Web & Frontend) ก่อนและหลังการปรับปรุง

- การเข้ารหัสเว็บ: ระบบบังคับใช้ HTTPS และ TLS 1.2/1.3 อยู่แล้วตั้งแต่ก่อนปรับปรุง และยังคงไว้หลังปรับปรุง
- รหัสลับ WordPress (Salts): ก่อนปรับปรุงใช้ค่าเริ่มต้นที่ซ้ำกัน เสี่ยงต่อการปลอมสิทธิ์ หลังปรับปรุงสุ่มคีย์ใหม่ยาว 64 ตัวอักษร จำนวน 8 ชุด
- กราฟบน Dashboard: ก่อนปรับปรุงนำ URL ไปแสดงบนกราฟโดยตรง เสี่ยงต่อการฝังโค้ดขโมยสิทธิ์ (Stored XSS) หลังปรับปรุงทำ Escape HTML ทุกจุด โค้ดที่แทรกเข้ามาจึงไม่ทำงาน
- นโยบายสคริปต์ (CSP): ก่อนปรับปรุงเปิด unsafe-inline ทำให้ยอมรับโค้ดที่แทรกเข้ามาได้ หลังปรับปรุงบังคับใช้ script-src 'self' เพื่อปิดการทำงานของโค้ดแทรก
- โฟลเดอร์รูปภาพ: ก่อนปรับปรุงเสี่ยงต่อการถูกแอบวาง WebShell หลังปรับปรุงตั้ง Nginx ให้บล็อกคำขอไฟล์ PHP ในโฟลเดอร์นี้ทันที และตอบกลับ HTTP 404

2) ความปลอดภัยระบบ และ เครือข่าย(System & Network)

#	จุดตรวจความปลอดภัยสำคัญ	ก่อนปรับปรุง (Before)	หลังปรับปรุง (After)
1	การเข้าสู่ระบบ SSH ปลอดภัย	[x] ปิดรหัสผ่านและ Root รับเฉพาะ Key	[x] ปิดรหัสผ่านและ Root รับเฉพาะ Key
2	ไฟร์วอลล์ IPv6 (IPv6 Bypass)	[ ] เปิดทุกพอร์ต เสี่ยงโดนเจาะเลี่ยงไฟร์วอลล์	[x] สั่ง DROP ปิดกั้นพอร์ตภายนอกทั้งหมด
3	สิทธิ์ของ Service ระบบ (Sandbox)	[ ] สิทธิ์ระบบกว้าง เข้าถึงอุปกรณ์ลึกเกินไป	[x] จำกัดสิทธิ์ Sandbox แยกการทำงานแต่ละตัว
4	การจัดเก็บ Log (Archive Integrity)	[ ] เซอร์วิสล้มเหลว ตรวจสอบประวัติไม่ได้	[x] กักกันไฟล์เสีย กู้คืนและบีบอัด 1,042 ไฟล์สำเร็จ
5	บันทึกประวัติระบบ (Audit Trail)	[ ] แอบแก้คอนฟิกระบบจะตรวจจับไม่ได้	[x] เปิด auditd เฝ้าระวังจุดสำคัญ 9 จุดตลอด 24 ชม.

รูปที่ 4.14 การตรวจสอบด้านระบบและเครือข่ายมีทั้งหมด 5 รายการ

- SSH: ปิดการเข้าสู่ระบบด้วยรหัสผ่าน และ root รับเฉพาะ Key อยู่แล้วตั้งแต่ก่อนปรับปรุง
- Firewall IPv6: ก่อนปรับปรุงเปิดทุกพอร์ต เสี่ยงต่อการถูกโจมตีเลี่ยงไฟร์วอลล์ทาง IPv6 หลังปรับปรุงตั้งให้ DROP เพื่อปิดกั้นพอร์ตจากภายนอกทั้งหมด
- สิทธิ์ของ Service (Sandbox): ก่อนปรับปรุงแต่ละ Service มีสิทธิ์กว้างและเข้าถึงอุปกรณ์ได้ลึกเกินจำเป็น หลังปรับปรุงจำกัดสิทธิ์ด้วย Sandbox และแยกการทำงานของแต่ละ Service ออกจากกัน
- การจัดเก็บ Log: ก่อนปรับปรุงเซอร์วิสล้มเหลวจนตรวจสอบประวัติไม่ได้ หลังปรับปรุงกักไฟล์ที่เสียไว้แยก กู้คืนและบีบอัดได้สำเร็จ 1,042 ไฟล์
- บันทึกประวัติระบบ (Audit Trail): ก่อนปรับปรุงหากมีการแอบแก้คอนฟิกระบบจะตรวจจับไม่ได้ หลังปรับปรุงเปิดใช้ auditd เฝ้าระวังจุดสำคัญ 9 จุดตลอด 24 ชั่วโมง

3) สิทธิ์การเข้าถึง และ ความเป็นส่วนตัว (Access & Data Privacy)

#	จุดตรวจความปลอดภัยสำคัญ	ก่อนปรับปรุง (Before)	หลังปรับปรุง (After)
1	การแยกเครือข่ายฐานข้อมูล	[x] MySQL ล็อกเฉพาะ 127.0.0.1	[x] MySQL ล็อกเฉพาะ 127.0.0.1
2	สิทธิ์ไฟล์คอนฟิก wp-config.php	[ ] สิทธิ์ 0644 คนในเครื่องแอบอ่านรหัส DB ได้	[x] ล็อกสิทธิ์ 0640 อ่านได้เฉพาะเว็บเซิร์ฟเวอร์
3	ปิดดึงรายชื่อผู้ใช้ (REST Users)	[ ] เรียก /wp/v2/users ดึงชื่อแอดมินได้ทันที	[x] บล็อก HTTP 403 และถอน Endpoint สำหรับคนนอก
4	สุ่มหาชื่อแอดมิน (?author=1)	[ ] เด้ง 301 เผยชื่อแอดมินใน URL ชัดเจน	[x] ตัดการเปลี่ยนทาง คืนค่า HTTP 404 Not Found
5	การส่งคีย์ Gemini API	[ ] แนบคีย์ใน URL เสี่ยงหลุดใน Log ระบบ	[x] ส่งผ่าน Header ลับ คีย์ไม่ปรากฏใน URL หรือ Log

รูปที่ 4.15 ผลการตรวจสอบด้านสิทธิ์การเข้าถึงและความเป็นส่วนตัว ก่อนและหลังการปรับปรุง

- ฐานข้อมูล: MySQL ล็อกให้รับการเชื่อมต่อเฉพาะ 127.0.0.1 อยู่แล้วตั้งแต่ก่อนปรับปรุง
- ไฟล์ wp-config.php: ก่อนปรับปรุงมีสิทธิ์ 0644 ทำให้ผู้ใช้อื่นในเครื่องแอบอ่านรหัสฐานข้อมูลได้ หลังปรับปรุงล็อกเป็น 0640 ให้อ่านได้เฉพาะเว็บเซิร์ฟเวอร์
- REST API /wp/v2/users: ก่อนปรับปรุงเรียกดูชื่อแอดมินได้ทันที (ดังรูปที่ 4.12) หลังปรับปรุงบล็อกด้วย HTTP 403 และถอน Endpoint นี้สำหรับคนนอก
- ?author=1: ก่อนปรับปรุงระบบ redirect แบบ 301 ทำให้เห็นชื่อแอดมินใน URL หลังปรับปรุงตัดการเปลี่ยนทางและคืนค่า HTTP 404 Not Found
- คีย์ Gemini API: ก่อนปรับปรุงแนบคีย์ไว้ใน URL เสี่ยงต่อการหลุดลง Log ของระบบ หลังปรับปรุงส่งผ่าน Header ลับ คีย์จึงไม่ปรากฏใน URL และ Log

4) การปิดกั้น URL และจุดรั่วไหลของระบบ (URL & Attack Surface)

#	จุดตรวจความปลอดภัยสำคัญ	ก่อนปรับปรุง (Before)	หลังปรับปรุง (After)
1	การตรวจสอบไฟล์ระบบหลัก	[x] แกนกลางระบบสะอาด ไม่พบมัลแวร์	[x] แกนกลางระบบสะอาด ไม่พบมัลแวร์
2	การดึงชื่อแอดมินผ่าน oEmbed	[ ] ส่ง author_url ชี้เป้าแอดมินในการ์ด Embed	[x] ตัดข้อมูลชื่อและลิงก์แอดมินออกจากการ์ดถาวร
3	สคริปต์ติดตั้งระบบ (install.php)	[ ] คนนอกเปิดหน้า install.php ได้ตรงๆ	[x] บล็อก HTTP 403 อนุญาตเฉพาะแอดมินล็อกอิน
4	ป้องกัน DoS ผ่าน wp-cron.php	[ ] บอทยิง HTTP Cron รัวๆ แย่งแรมจนเว็บค้าง	[x] รันผ่าน Linux CLI ทุก 15 นาที และบล็อก HTTP
5	ไฟล์คู่มือและระบบ (readme.html)	[ ] เปิดอ่าน readme.html ได้ เผยเลขเวอร์ชัน	[x] Nginx บล็อกคืนค่า HTTP 404 Not Found ทันที

รูปที่ 4.16 ผลการตรวจสอบด้านการปิดกั้น URL และจุดรั่วไหลของระบบ ก่อนและหลังการปรับปรุง

- ไฟล์ระบบหลัก: แกนกลางระบบสะอาด ไม่พบมัลแวร์ ทั้งก่อนและหลังปรับปรุง
- oEmbed: ก่อนปรับปรุงการ์ด Embed ส่ง author_url ที่ชี้ไปยังแอดมิน หลังปรับปรุงตัดข้อมูลชื่อและลิงก์แอดมินออกจากการ์ด Embed อย่างถาวร
- install.php: ก่อนปรับปรุงคนนอกเปิดหน้านี้ได้โดยตรง หลังปรับปรุงบล็อกด้วย HTTP 403 และอนุญาตเฉพาะแอดมินที่ล็อกอินแล้ว
- wp-cron.php: ก่อนปรับปรุงบอทยิง HTTP Cron ถี่ ๆ จนแย่งทรัพยากรและเว็บค้าง หลังปรับปรุงเปลี่ยนไปรันผ่าน Linux CLI ทุก 15 นาที และบล็อกการเรียกผ่าน HTTP
- readme.html: ก่อนปรับปรุงเปิดอ่านได้และเผยเลขเวอร์ชันของระบบ หลังปรับปรุงตั้ง Nginx ให้คืนค่า HTTP 404 Not Found ทันที

บทที่ 5 สรุปผลและข้อเสนอแนะ

5.1 สรุปผลการดำเนินโครงงาน

โครงงานนี้พัฒนาเว็บไซต์ประชาสัมพันธ์หลักสูตรวิทยาการคอมพิวเตอร์ด้วย WordPress บน Azure VPS ได้สำเร็จตามวัตถุประสงค์ โดยสรุปผลแยกตามระยะของการดำเนินงานได้ดังนี้

ระยะที่ 1 (Stage 1) ระบบให้บริการเว็บไซต์ผ่าน HTTPS ที่โดเมน networkkeroro.duckdns.org ด้วยใบรับรองจาก Let's Encrypt ซึ่งต่ออายุอัตโนมัติด้วย DNS-01 Challenge จึงไม่ต้องเปิดพอร์ต 80 ให้ต่างประเทศ ด้านความปลอดภัย ระบบป้องกันหลายชั้นตั้งแต่ Azure NSG และ Firewall ด้วย iptables ร่วมกับ ipset ที่รับการเชื่อมต่อเฉพาะ IP ประเทศไทย ผลการทดสอบผ่าน check-host.net ยืนยันว่าเครื่องตรวจสอบในต่างประเทศไม่สามารถเชื่อมต่อพอร์ต 22 และ 443 ได้ ส่วนการเข้าถึง SSH ต้องใช้ Private Key ร่วมกับรหัส TOTP เท่านั้น การทดสอบยืนยันว่าระบบปฏิเสธทั้งกรณีรหัส TOTP ผิด กรณีไม่มี Private Key และการเข้าสู่ระบบด้วยบัญชี root

ระยะที่ 2 (Stage 2) ระบบได้รับการปรับให้ใช้ Reverse Proxy แยก Nginx Frontend ออกจาก Backend ที่รับการเชื่อมต่อเฉพาะภายในเครื่อง มีการจัดเก็บ Security Log แบบ JSON และกำหนด Rate Limit แยกตาม Endpoint พร้อมระบบวิเคราะห์ภัยคุกคามอัตโนมัติด้วย Google Gemini API ที่ควบคุมด้วย Data Masking, Validator, Security Rules และ Fallback ผลการวิเคราะห์จากทราฟฟิกจริงจำนวน 35,130 คำขอ ระบบตรวจพบเหตุการณ์ด้านความปลอดภัย 782 เหตุการณ์ เป็นระดับความเสี่ยงสูง 124 เหตุการณ์ โดยภัยที่พบมากที่สุดคือการพยายามสุ่มรหัสผ่านผ่าน /wp-login.php ผลทั้งหมดแสดงบน Security Dashboard พร้อมเหตุผลภาษาไทย และเชื่อมกับระบบบล็อก IP อัตโนมัติ การปลดบล็อกด้วย OTP และการแจ้งเตือนทางอีเมล โดยภาพรวม ระบบทำงานครอบคลุมข้อกำหนดของทั้ง 2 ระยะ และแสดงให้เห็นว่าการป้องกันหลายชั้นตามหลัก Defense in Depth ร่วมกับการเฝ้าระวังด้วย AI ช่วยให้ผู้ดูแลระบบเห็นภาพและตอบสนองต่อภัยคุกคามได้รวดเร็วขึ้น

5.2 ปัญหา อุปสรรค และแนวทางแก้ไข

ระหว่างการพัฒนาและทดสอบระบบ คณะผู้จัดทำพบปัญหาที่สำคัญและได้แก้ไขดังนี้

5.2.1 การต่ออายุใบรับรองขัดกับนโยบาย Firewall

วิธี HTTP-01 Challenge ของ Let’s Encrypt ต้องให้เซิร์ฟเวอร์ตรวจสอบในต่างประเทศเชื่อมต่อเข้ามาที่พอร์ต 80 ซึ่งถูก Firewall ปฏิเสธ คณะผู้จัดทำจึงเปลี่ยนไปใช้ DNS-01 Challenge ผ่าน TXT Record ของ DuckDNS ทำให้ต่ออายุใบรับรองได้อัตโนมัติโดยไม่ต้องเปิดพอร์ตให้ต่างประเทศ

5.2.2 ความเสี่ยงต่อการล็อกตัวเองออกจากเซิร์ฟเวอร์

การแก้ไขการตั้งค่า SSH และ PAM ผิดพลาดอาจทำให้ผู้ดูแลเข้าเซิร์ฟเวอร์ไม่ได้อีก จึงกำหนดขั้นตอนให้สำรองไฟล์ก่อนแก้ไข ตรวจไวยากรณ์ด้วย sshd -t และทดสอบจาก Session ใหม่ก่อนปิด Session เดิมทุกครั้ง

5.2.3 SSH ถามรหัสผ่าน Linux เพิ่มหลังเปิดใช้ TOTP

สาเหตุเกิดจากบรรทัด @include common-auth ในไฟล์ PAM ของ SSH ที่เรียกการยืนยันตัวตนด้วยรหัสผ่านอยู่ด้วย คณะผู้จัดทำจึงปิดบรรทัดดังกล่าว ทำให้การเข้าสู่ระบบใช้เพียง Private Key ร่วมกับรหัส TOTP ตามที่ออกแบบ

5.2.4 Dashboard เรียกข้อมูลของตัวเองซ้ำ (Self-Polling)

Dashboard เรียกไฟล์ analysis.json ถี่เพื่ออัปเดตข้อมูล จึงถูก Rate Limit และสร้าง Log ที่ไม่จำเป็นเข้าสู่การวิเคราะห์ แก้ไขโดยยกเว้น analysis.json จาก Rate Limit และ Security Log แต่ยังคงป้องกันด้วย Basic Authentication

5.2.5 Rate Limit ค่าเดียวกระทบผู้ใช้ปกติ

การเปิดหน้าเว็บหนึ่งหน้าต้องโหลดไฟล์ประกอบจำนวนมาก การใช้ค่าเดียวทั้งระบบจึงทำให้ผู้ใช้ทั่วไปถูกจำกัดได้ง่าย คณะผู้จัดทำจึงแยก Policy ตาม Endpoint โดยเข้มงวดที่หน้า Login และยกเว้นไฟล์ Static

5.2.6 การอ่านไฟล์ผลวิเคราะห์ขณะกำลังเขียน

Dashboard และบริการบล็อก IP อาจอ่านไฟล์ระหว่างที่ Pipeline ยังเขียนไม่เสร็จ ทำให้ได้ข้อมูลไม่สมบูรณ์ แก้ไขด้วย Atomic Write คือเขียนลงไฟล์ชั่วคราวให้เสร็จก่อนแล้วจึงเปลี่ยนชื่อทับไฟล์เดิม

5.2.7 Gemini API ติดโควตาหรือเชื่อมต่อไม่ได้

ระบบวิเคราะห์พึ่งพาบริการภายนอก หาก API ขัดข้อง การเฝ้าระวังจะหยุดทำงาน จึงออกแบบให้ใช้ API Key หลายชุด สลับโมเดล ใช้ Cache และมี Deterministic Fallback ที่วิเคราะห์ด้วยกฎแทนได้ทันที

5.2.8 ผลทดสอบ Firewall ผิดปกติจากโหนด Kyiv

การทดสอบผ่าน check-host.net พบโหนด Kyiv แสดงผล Connected ขณะที่ประเทศอื่นถูกปฏิเสธทั้งหมด เมื่อตรวจสอบเพิ่มพบว่าโหนดดังกล่าวตอบรับเร็วผิดปกติ (0.003 วินาที) และตอบรับแม้ที่พอร์ต 9999 ซึ่งไม่ได้เปิดใช้งาน จึงสรุปได้ว่าโหนดตอบรับเอง ไม่ได้เชื่อมต่อถึงเซิร์ฟเวอร์จริง

5.3 ข้อจำกัดของระบบ

แม้ระบบจะทำงานได้ตามข้อกำหนด แต่จากการพัฒนาและทดสอบพบข้อจำกัดที่ควรพิจารณา ดังนี้

- การจำกัด IP ตามประเทศป้องกันได้ไม่ทั้งหมด ผลจาก Dashboard แสดงว่าเหตุการณ์จำนวนมากมาจาก IP ในประเทศไทยซึ่งผ่าน Firewall ได้ตามปกติ และผู้โจมตีสามารถใช้ VPN หรือเครื่องในประเทศไทยหลบเลี่ยงได้ นอกจากนี้ รายการ IP จาก IPdeny อาจไม่ครบถ้วนหรือไม่เป็นปัจจุบัน
- ผู้ใช้ในต่างประเทศเข้าถึงเว็บไซต์ไม่ได้ ซึ่งเป็นข้อแลกเปลี่ยนจากนโยบาย Geo-blocking และระบบรองรับเฉพาะ IPv4
- การตอบสนองมีความหน่วง การวิเคราะห์จาก Log และรอบการทำงานของ Timer ทำให้ระบบเป็นการตรวจจับและตอบสนองหลังเกิดเหตุ ไม่ใช่การป้องกันแบบทันที การบล็อก IP จึงมีผลกับคำขอถัดไปเท่านั้น
- การบล็อกตาม IP อาจกระทบผู้ใช้ปกติ กรณีผู้ใช้หลายคนใช้ IP เดียวกันผ่าน NAT เช่น เครือข่ายมหาวิทยาลัยหรือเครือข่ายมือถือ
- ระบบวิเคราะห์พึ่งพาบริการภายนอก ทั้งด้านโควตาและความพร้อมใช้งานของ Gemini API และผลวิเคราะห์ของ LLM อาจคลาดเคลื่อนได้ จึงต้องมี Validator และ Security Rules ควบคุม
- ยังไม่ได้วัดความแม่นยำเชิงปริมาณ เนื่องจากยังไม่มีชุดข้อมูลทดสอบที่ติดป้ายกำกับ จึงยังไม่สามารถระบุค่า Precision หรือ Recall ของการตรวจจับได้
- ระบบทำงานบน VPS เครื่องเดียว จึงยังมีจุดล้มเหลวเดียว (Single Point of Failure)

5.4 แนวทางพัฒนาต่อ

เพื่อให้ระบบมีความสมบูรณ์และน่าเชื่อถือมากขึ้น คณะผู้จัดทำเสนอแนวทางพัฒนาต่อ ดังนี้

- ปิดหรือจำกัด /xmlrpc.php เนื่องจากผลจาก Dashboard พบการเรียกเส้นทางนี้ 220 ครั้ง และเว็บไซต์ไม่ได้ใช้งาน XML-RPC
- ป้องกันการค้นหาชื่อผู้ใช้ (User Enumeration) โดยจำกัดการเข้าถึง /wp-json/wp/v2/users และ /?author= ซึ่งพบการเรียกในผลการวิเคราะห์
- เพิ่มการยืนยันตัวตนสองปัจจัยให้หน้า /wp-admin เนื่องจาก /wp-login.php เป็นเป้าหมายที่ถูกโจมตีมากที่สุด (3,856 ครั้ง)
- จัดทำชุดข้อมูลทดสอบที่ติดป้ายกำกับ ประกอบด้วยทราฟฟิกปกติและการโจมตีจำลอง แล้วประเมินประสิทธิภาพตามตารางที่ 5.2
- ตั้งเวลาปรับปรุงรายการ IP ประเทศไทยอัตโนมัติ และรองรับ IPv6
- เพิ่ม Web Application Firewall (WAF) เช่น ModSecurity ร่วมกับ OWASP Core Rule Set เพื่อบล็อกคำขอที่เป็นอันตรายได้ทันทีโดยไม่ต้องรอผลวิเคราะห์
- สำรองข้อมูลอัตโนมัติ ทั้งฐานข้อมูลและไฟล์เว็บไซต์ไปยังพื้นที่จัดเก็บภายนอก และเพิ่มการตรวจสุขภาพของ Service พร้อมแจ้งเตือนเมื่อ Pipeline หยุดอัปเดต
- เพิ่มช่องทางแจ้งเตือน เช่น LINE หรือ Discord และส่ง Log ไปยังระบบ SIEM เพื่อวิเคราะห์ย้อนหลัง

บรรณานุกรม

[1] Microsoft. *Azure network security groups overview*. URL: https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview (visited on 10/28/2026).

[2] Canonical. *Ubuntu Server documentation*. URL: https://ubuntu.com/server/docs/ (visited on 10/28/2026).

[3] F5 NGINX. *NGINX Reverse Proxy*. URL: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy (visited on 10/28/2026).

[4] WordPress.org. *Requirements*. URL: https://wordpress.org/about/requirements/ (visited on 10/28/2026).

[5] Duck DNS. *HTTP API Specification*. URL: https://www.duckdns.org/spec.jsp (visited on 10/28/2026).

[6] Let's Encrypt. *Challenge Types*. URL: https://letsencrypt.org/docs/challenge-types/ (visited on 10/28/2026).

[7] OpenSSH. *OpenSSH Manual Pages*. URL: https://www.openssh.org/manual.html (visited on 10/28/2026).

[8] D. M'Raihi, S. Machani, M. Pei, and J. Rydell. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238. Informational. RFC Editor, May 2011. DOI: 10.17487/RFC6238. URL: https://www.rfc-editor.org/rfc/rfc6238.html (visited on 10/28/2026).

[9] The Netfilter Project. *Documentation about the netfilter/iptables project*. URL: https://netfilter.org/documentation/ (visited on 10/28/2026).

[10] The Netfilter Project. *The netfilter.org ipset project*. URL: https://www.netfilter.org/projects/ipset/index.html (visited on 10/28/2026).

[11] IPdeny. *IPdeny IP country blocks*. URL: https://www.ipdeny.com/ (visited on 10/28/2026).

[12] Fail2Ban. *Fail2Ban: Daemon to ban hosts that cause multiple authentication errors*. URL: https://github.com/fail2ban/fail2ban (visited on 10/28/2026).

[13] NGINX. *Module ngx_http_limit_req_module*. URL: https://nginx.org/en/docs/http/ngx_http_limit_req_module.html (visited on 10/28/2026).

[14] NGINX. *Module ngx_http_log_module*. URL: https://nginx.org/en/docs/http/ngx_http_log_module.html (visited on 10/28/2026).

[15] Google. *Structured outputs*. URL: https://ai.google.dev/gemini-api/docs/structured-output (visited on 10/28/2026).

[16] systemd Project. *systemd.timer — Timer unit configuration*. URL: https://freedesktop.org/software/systemd/man/latest/systemd.timer.html (visited on 10/28/2026).

[17] Cloudflare. *security-audit-skill*. Code repository. GitHub. URL: https://github.com/cloudflare/security-audit-skill (visited on 10/29/2026).